



ENGENHARIA DE SOFTWARE

Conteúdo

- ? Conceitos
- ? Ciclo de vida
- ? Modelos de software
- ? Métodos ágeis
- ? Prototipação
- ? Levantamento de Requisitos
- ? Projeto de Software
- ? Manutenção
- ? Testes



1

Introdução

Conceito

O que é Engenharia de Software?

A engenharia de software é uma das áreas da computação que apoia o desenvolvimento de software profissional.

Suas técnicas orientam a especificação, o desenvolvimento, a validação e a evolução do software.



O que é Engenharia de Software?

Os engenheiros fazem as coisas funcionarem com a aplicação de técnicas, métodos e ferramentas. Esses elementos são usados seletivamente e eles sempre tentam descobrir soluções para os problemas.

A engenharia de software se preocupa com os processos técnicos do desenvolvimento, mas também com gerenciamento de projetos, ferramentas, métodos e teorias.



História da Engenharia de software

O conceito da engenharia de software surgiu pela primeira vez em 1968 em uma conferência que discutia a crise do software.

Nos anos 1970 e 1980, foram desenvolvidas técnicas e métodos de engenharia de software como a programação estruturada, a ocultação de informação (*information hiding*) e o desenvolvimento orientado a objetos. As ferramentas e notações desenvolvidas nesta época é a base utilizada até hoje.



O que é software?

Caracteriza-se por programas de computador e suas respectivas documentações.

Os entregáveis são destinados aos clientes ou para um mercado genérico.



O que seria um bom software?

Podemos dizer que o bom software é aquele que fornece as funcionalidades e funções que o cliente necessita com o desempenho adequado.

Ciência da Computação x Engenharia de Software

A ciência da computação tem seu foco na teoria e nos fundamentos dos assuntos de TI. Já a engenharia de software foca nas questões práticas para desenvolver e entregar um software útil.



Engenharia de Software

A Engenharia de Software é um conceito macro/geral. Ela está presente desde quando surge a ideia de software até quando ele “morre”.

O objetivo aqui é orientar, guiar, acompanhar e prover todos os elementos necessários para que os programas sejam úteis.



Desenvolvimento: Profissional x Amador

PROFISSIONAL

Vários módulos ou programas

Parametrizações específicas para os módulos

Manual de utilização

Documentação técnica e funcional

AMADOR

Específico para um determinado contexto

Ausência de parametrizações ou parametrizações gerais

Ausência de manual

Ausência de documentação e comentários

Tipos de produto de software

Produtos genéricos

São sistemas desenvolvidos para qualquer cliente que deseje compra-los. Exemplos: aplicativos para dispositivos móveis e softwares para computador (bancos de dados, processadores de texto, pacotes de desenho, etc). Geralmente são focados para um mercado específico como bibliotecas, odontologia, contabilidade e outros.

Software personalizado (sob medida):

São sistemas encomendados e desenvolvidos especificamente para um determinado cliente. É um trabalho "artesanal".

Tipos de produto de software

Apesar desta definição/separação teórica, nos dias de hoje há uma certa dificuldade na distinção do software, pois estão cada vez mais comuns produtos genéricos que podem ser adaptados para as necessidades do cliente, são os ERPs, *Enterprise Resource Planning* (Planejamento de Recursos Empresariais). As empresas SAP e Oracle fornecem esses tipos de sistema.



Atributos essenciais de um bom software

Aceitabilidade

- Ter boa aceitação com o usuário a que ele se destina. Deve ser útil, prático e compatível com outros sistemas que esse usuário utiliza.

Dependabilidade e Segurança

- Ser confiável e seguro. O software com dependabilidade não deve causar danos físicos ou econômicos em caso de falha do sistema. Também deve ser protegido de usuários maliciosos.

Eficiência

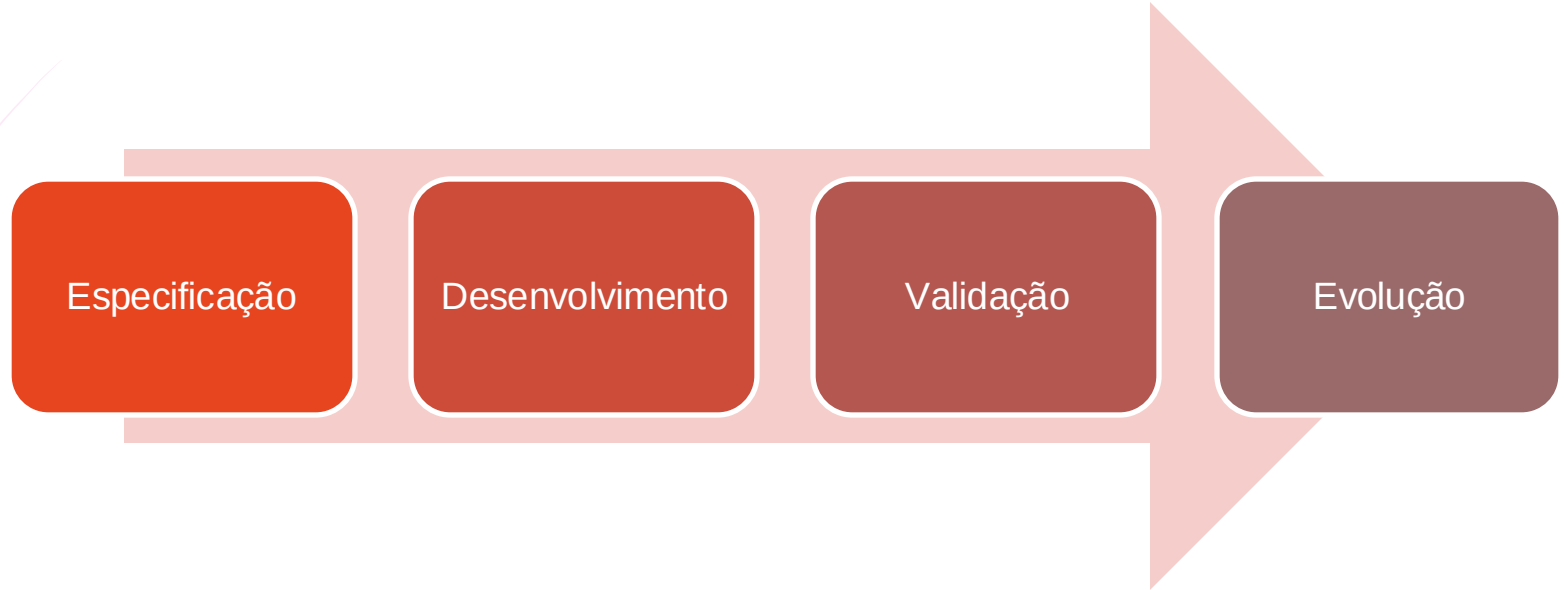
- Não deve desperdiçar recursos como memória e processamento. Portanto, a eficiência inclui responsividade, tempo de processamento, utilização de recursos e etc.

Manutenibilidade

- Deve ser escrito de modo que possa evoluir e atender as necessidades do usuário a medida que elas vão mudando, ele deve ser adaptável.

Processo de Software

Uma abordagem utilizada pela Engenharia de Software é chamada de Processo de Software que se traduz em uma sequência de atividades que levam a produção de um software.



Processo de Software

- ▶ **Especificação:** é a etapa onde os clientes e engenheiros definem o software que deve ser produzido e as restrições impostas à sua operação.
- ▶ **Desenvolvimento:** é a etapa onde o software é projetado e programado.
- ▶ **Validação:** é a etapa em que o programa é analisado para garantir que seja aquilo que o cliente precisa.
- ▶ **Evolução:** é a etapa onde se reflete sobre modificações dos requisitos tanto do cliente quanto do mercado.

Tipos de Aplicação

Stand-Alone

- São os sistemas que rodam em computadores ou aplicativos que rodam em dispositivos móveis.

Interativas baseadas em transações

- São os sistemas que rodam em um computador remoto e que são acessadas pelos usuários a partir de seus próprios computadores, tablets ou smartphones.

Sistemas de controle embarcados

- São sistemas de controle de software que controlam e gerenciam dispositivos de hardware.

Sistemas de processamento em lotes

- São sistemas de negócio concebidos para processar dados em grandes lotes.

Tipos de Aplicação

Entretenimento

- São os sistemas destinados a uso pessoal, para entreter o usuário

Modelagem e Simulação

- São os sistemas desenvolvidos por engenheiros e cientistas para modelar processos físicos ou situações que incluem muitos objetos diferentes e que interagem.

Coleta de dados e análise

- São sistemas que fazem a coleta de dados no ambiente e enviam esses dados para outros sistemas para processamento.

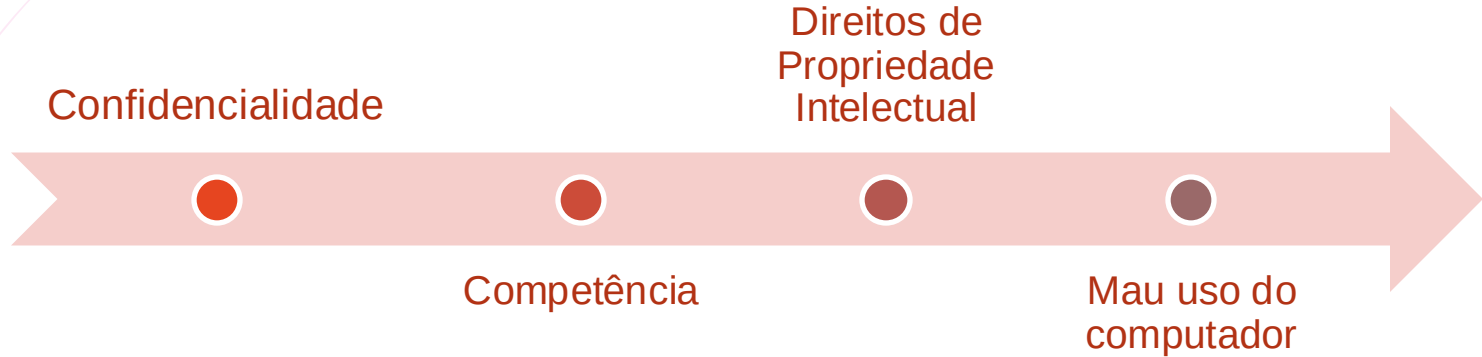
Sistemas de sistemas

- São sistemas utilizados em empresas e organizações e são compostos de uma série de outros sistemas de software.

O Engenheiro de Software

Assim como outras disciplinas de engenharia, a engenharia de software é conduzida por uma série de parâmetros sociais e legais que delimitam os trabalhos nesse setor.

Desta forma, o engenheiro de software deve agregar algumas qualidades profissionais:





2

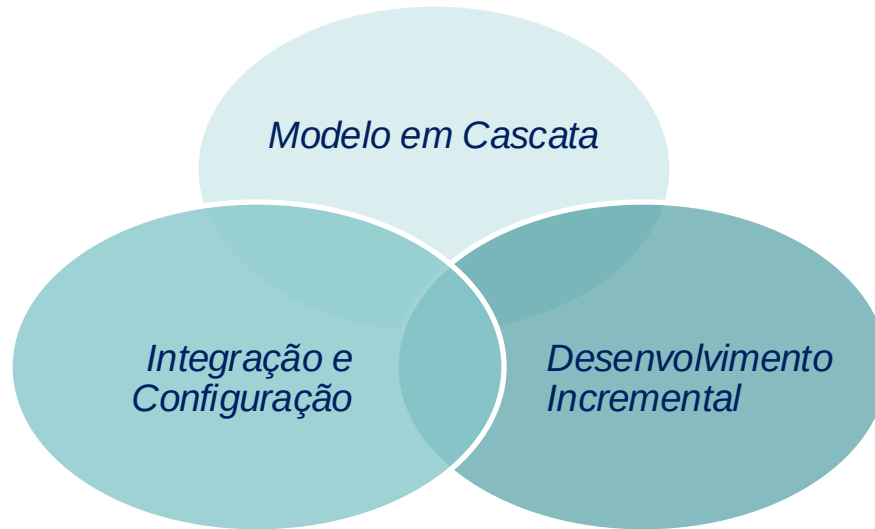
Ciclo de Vida

Modelos de Processos de Software

Modelos de Processos de Software

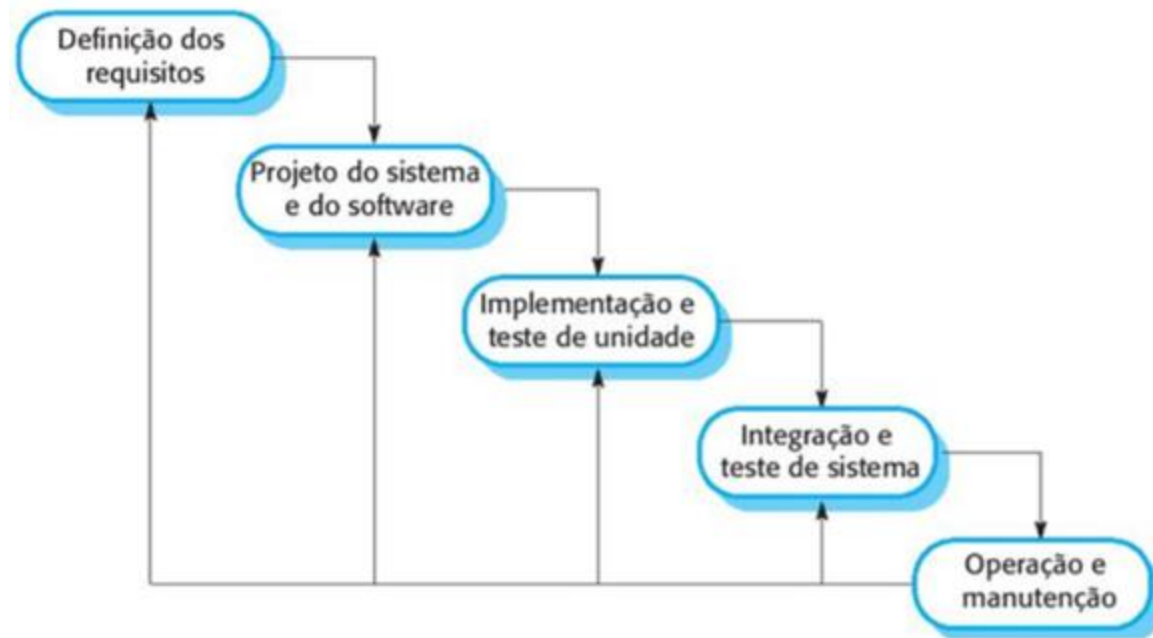
Os modelos de processos de software, também chamado de Ciclo de Vida do desenvolvimento de software é uma representação simplificada de um processo de software.

Podemos destacar alguns modelos genéricos que podem ser ampliados ou adaptados:



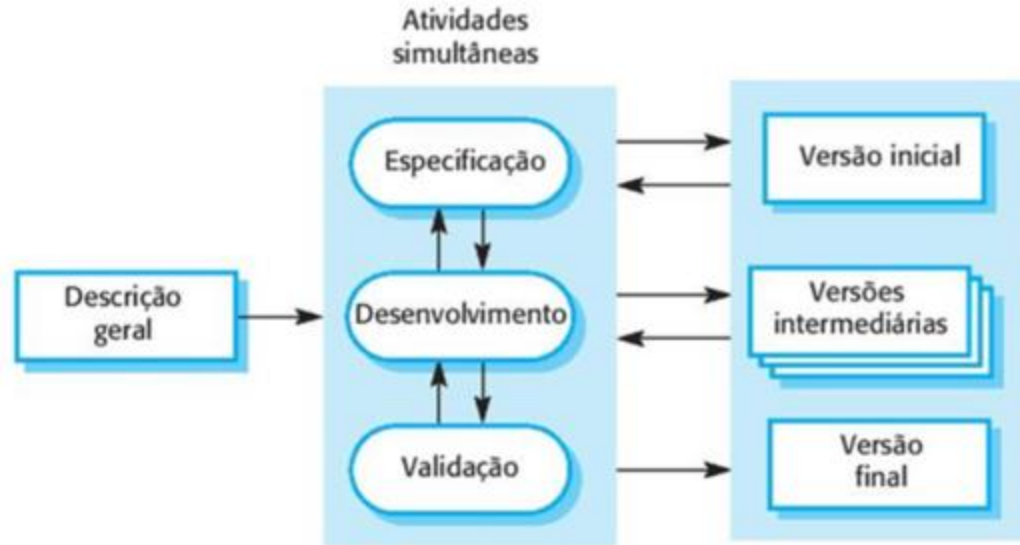
Modelo em Cascata

O modelo em cascata foi baseado em modelos de engenharia de grandes sistemas militares. Ele apresenta o processo de desenvolvimento de software como uma série de estágios.



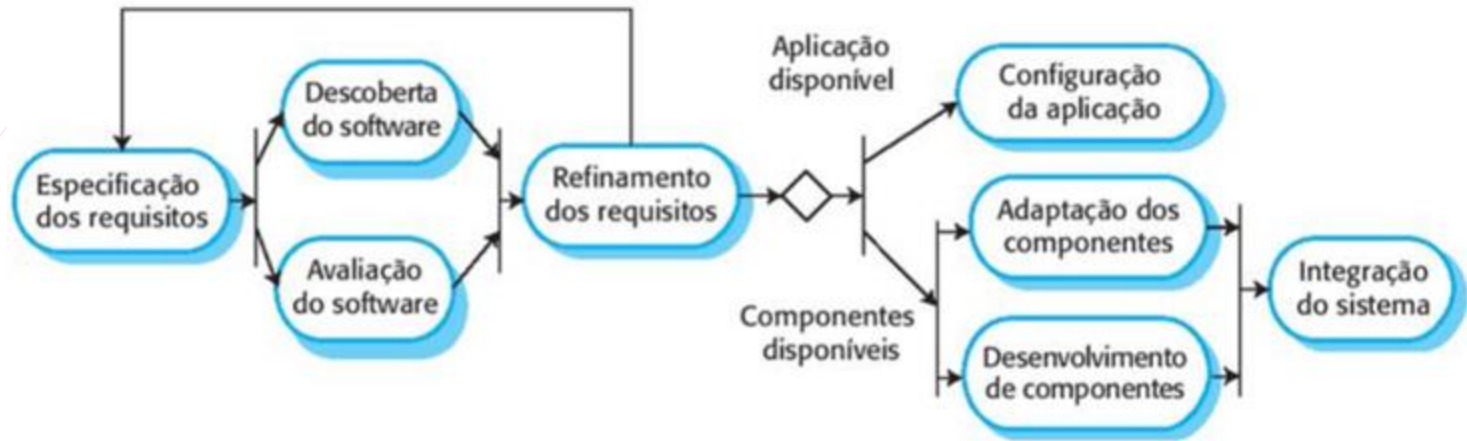
Desenvolvimento Incremental

O desenvolvimento incremental se baseia na ideia de desenvolver uma implementação inicial, obter feedback e fazer o software evoluir através de várias versões. A especificação, desenvolvimento e validação são intercaladas, em vez de separadas, e com o feedback rápido ao longo de todas elas.



Integração e Configuração

Na maioria dos projetos há reuso de software. Os desenvolvedores procuram um código similar ao necessário, fazem as modificações adequadas e integram código do novo software.



Exemplos

Sistemas críticos em segurança

- Normalmente desenvolvidos pelo Modelo Cascata, pois precisa de uma grande quantidade de análise e documentação.

Produtos de software

- Atualmente desenvolvidos pelo modelo incremental

Sistemas de negócio

- Cada vez mais desenvolvidos por meio de configuração de sistema preexistentes e da integração entre eles



Não há um modelo de processo universal!

Tudo vai depender:

1. Do cliente;
2. Dos requisitos do software;
3. Do ambiente em que o software será utilizado;
4. Tipo de software que será desenvolvido.

Modelos de Processo de Software

RUP

Apesar de ainda não existir um modelo universal, há vários estudos direcionados a esse intuito. Um dos mais conhecidos é o RUP, *Rational Unified Process*, que foi desenvolvido por uma empresa de engenharia de software norte-americana, a Rational.

Trata-se de um modelo flexível que pode ser chamado de diferentes maneiras para criar processos que se assemelhem a qualquer um dos modelos genéricos.

Foi adotado por grandes empresas como a IBM, por exemplo, mas enfrentou muita resistência e pouca aceitação.

Modelos de Processo de Software

RUP

Normalmente utiliza 3 perspectivas:

Dinâmica

- Mostra as fases do modelo no tempo

Estática

- Mostra as atividades do processo

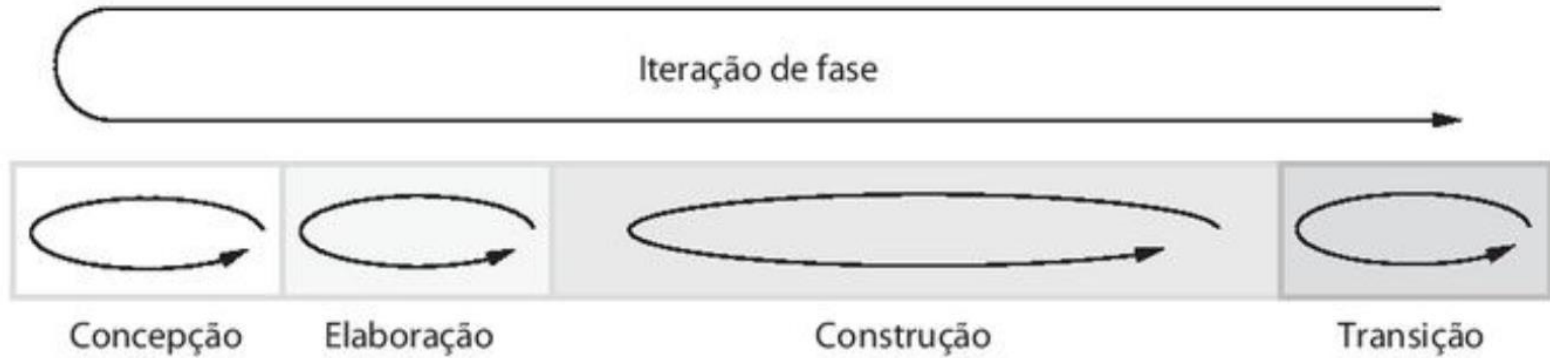
Prática

- Sugere práticas a serem utilizadas no processo.

Modelos de Processo de Software

RUP

Fases no Rational Unified Process



Modelos de Processo de Software

PONTOS IMPORTANTES

- Os processos de software são as atividades envolvidas na produção de um sistema de software. Modelos de processos de software são representações abstratas desses processos.
- Modelos gerais de processo descrevem a organização dos processos de software. Exemplos desses modelos gerais incluem o modelo em cascata, o desenvolvimento incremental e o desenvolvimento orientado a reuso.
- Engenharia de requisitos é o processo de desenvolvimento de uma especificação de software. As especificações destinam-se a comunicar as necessidades de sistema dos clientes para os desenvolvedores do sistema.
- Processos de projeto e implementação estão relacionados com a transformação das especificações dos requisitos em um sistema de software executável. Métodos sistemáticos de projeto podem ser usados como parte dessa transformação.
- Validação de software é o processo de verificação de que o sistema está de acordo com sua especificação e satisfaz às necessidades reais dos usuários do sistema.
- Evolução de software ocorre quando se alteram os atuais sistemas de software para atender aos novos requisitos. As mudanças são contínuas, e o software deve evoluir para continuar útil.
- Processos devem incluir atividades para lidar com as mudanças. Podem envolver uma fase de prototipação, que ajuda a evitar más decisões sobre os requisitos e projeto. Processos podem ser estruturados para o desenvolvimento e a entrega iterativos, de forma que mudanças possam ser feitas sem afetar o sistema como um todo.
- O Rational Unified Process (RUP) é um moderno modelo genérico de processo, organizado em fases (concepção, elaboração, construção e transição), mas que separa as atividades (requisitos, análises, projeto etc.) dessas fases.



3

Prototipação

Representação gráfica de software

Prototipação

O protótipo é uma versão inicial do sistema utilizado para demonstrar conceitos, experimentar opções de projeto, descobrir mais sobre o problema e suas possíveis soluções.

Um protótipo de software pode ser utilizado em um processo de desenvolvimento para ajudar a antecipar as mudanças que podem ser necessárias durante o levantamento de requisitos e no projeto do sistema.



Prototipação

Não temos uma "receita de bolo" para criar protótipos, a ideia aqui é não limitar a criação. No entanto, a literatura nos indica alguns tipos de protótipos que podem nos dar uma base para avançar.

STORYBOARDS

São histórias contadas em forma de quadros em uma linha do tempo.

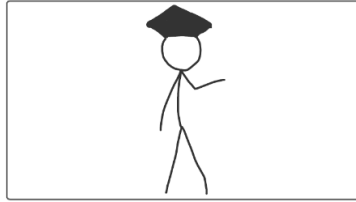
É uma excelente ferramenta para descrever uma ideia de relação de um usuário com um produto.



Tutatamafilm/Shutterstock

Prototipação - StoryBoards

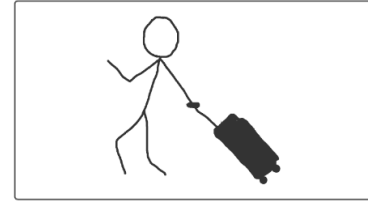
Se a equipe não tiver um colaborador com habilidade em desenho, utilizar o "boneco palito" é uma alternativa para efetuar a representação. Outra abordagem é utilizar ferramentas específicas.



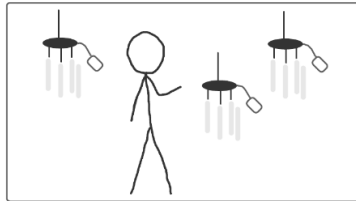
Main person is at their graduation ceremony.



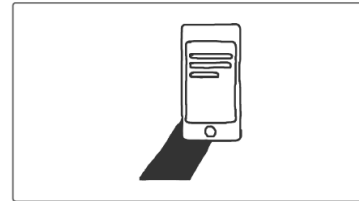
After graduation, the graduate packs bags to go to Morocco.



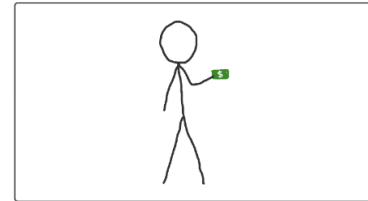
The flight lands in Morocco and the graduate is walking out of the airport very excited.



The graduate wants to have an authentic experience and goes shopping at the local bazaar. The wind chimes particularly stand out.



Graduate uses our app to translate from English to Arabic.



Graduate makes a purchase.

Prototipação - StoryBoards

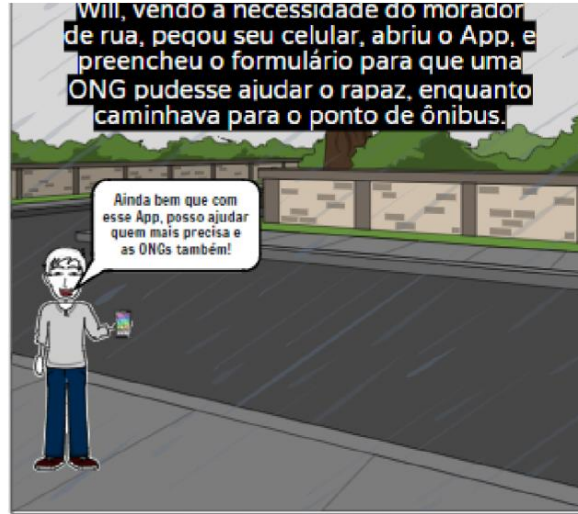


DIAGRAMA OU MODELAGEM

Basicamente o diagrama é a materialização de um conceito que está abstrato em forma de desenho e que pode ser apresentado em folhas de papel, murais, aplicativos de apresentação ou em softwares.

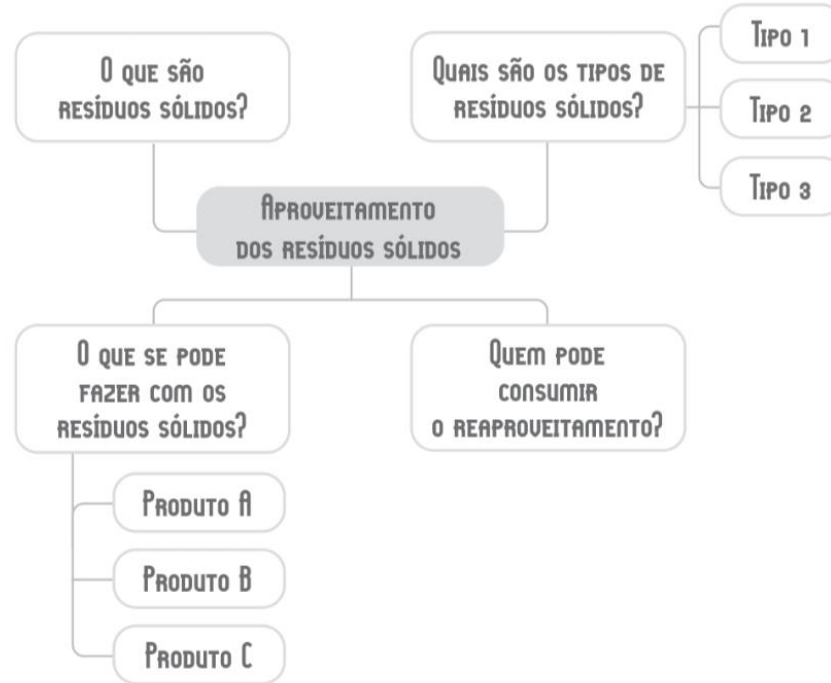
O diagrama mais conhecido é o fluxograma. Veja um exemplo:

Prototipação



Prototipação – Diagrama ou Modelagem

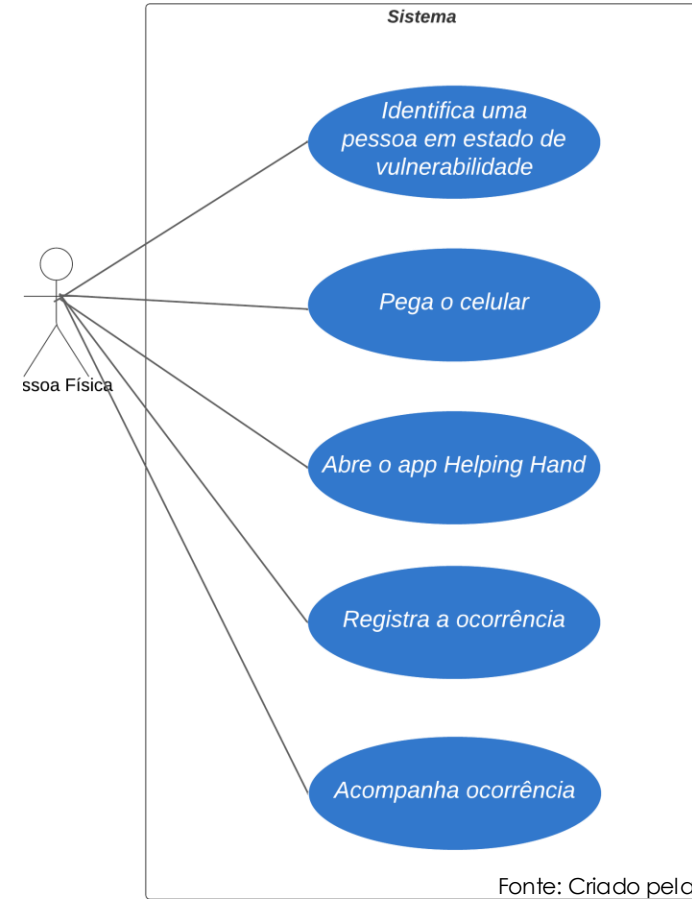
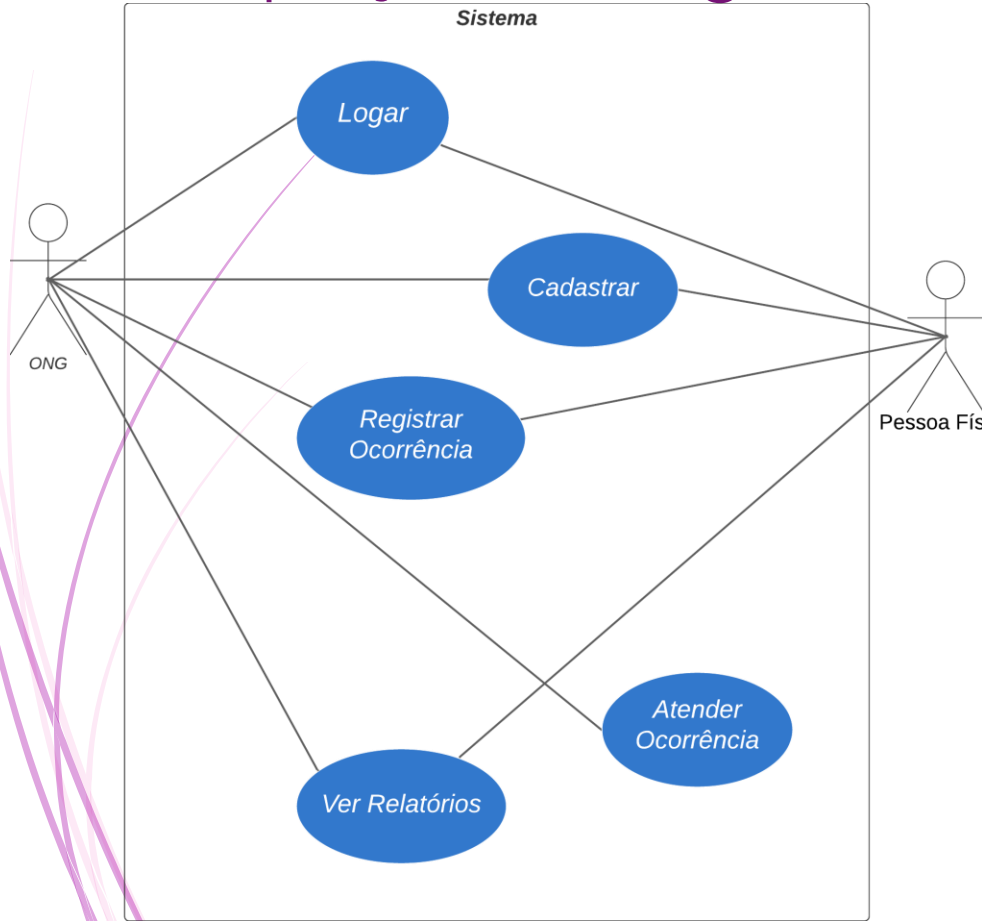
Outro exemplo de diagrama que pode ser citado é o Mapa Mental, no qual as ideias são organizadas por tópicos sendo marcadas as relações entre os elementos.

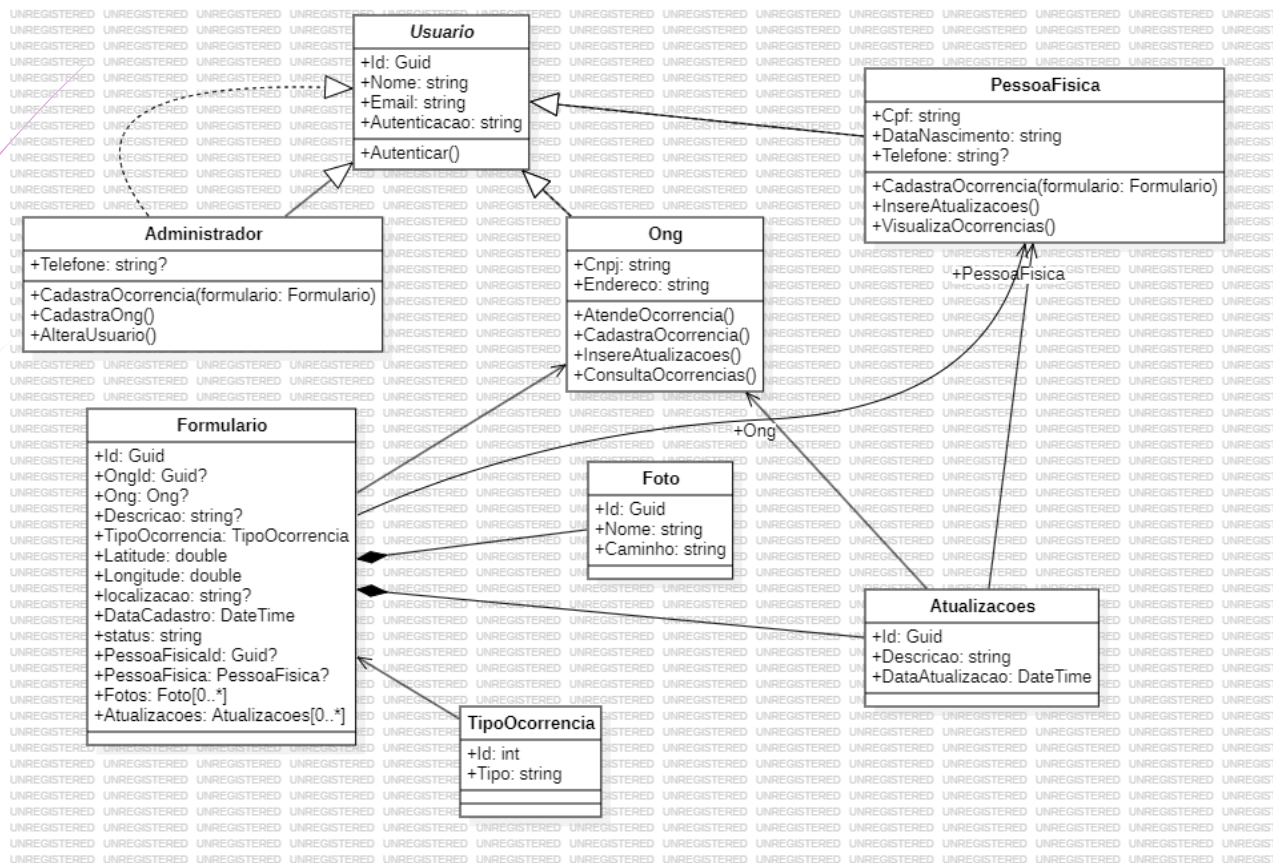


Prototipação – Diagrama ou Modelagem



Prototipação – Diagrama ou Modelagem





NARRATIVA

É um recurso que permite contar uma história, a prática pode resultar em deduções pertinentes à elaboração de conceitos sobre um assunto e engajar pessoas.

A narrativa precisa conter algumas perguntas a serem respondidas podendo nortear sua construção.

Pra quem?

- Público a quem se destina o produto ou processo.

O quê?

- Indica qual ou quais recursos serão utilizados pelo personagem (público-alvo).

Porquê?

- Indica o motivo que faz o personagem (público-alvo) querer esse produto.

Prototipação

PUBLICIDADE

A publicidade é uma forma de apresentar uma ideia que existe há um tempo. Também chamada de propaganda, implica levar ao conhecimento do público algo inusitado ou que lhe trará benefícios.

HstrongART/Shutterstock



MODELOS EM PAPEL

Uma técnica mais simplista e comum é criar protótipos em papel utilizando lápis ou caneta!



Chaosamran_Studio/Shutterstock

MAQUETE

As maquetes comunicam uma ideia de tridimensionalidade do produto. Vários tipos de materiais podem ser utilizados para sua construção, mas os mais indicados são os recicláveis.

O primeiro resultado pode não ser o mais adequado e agradável, mas a proposta é ir refinando a ideia até chegar no mais próximo possível do que se pretende construir/desenvolver.

ENCENAÇÃO

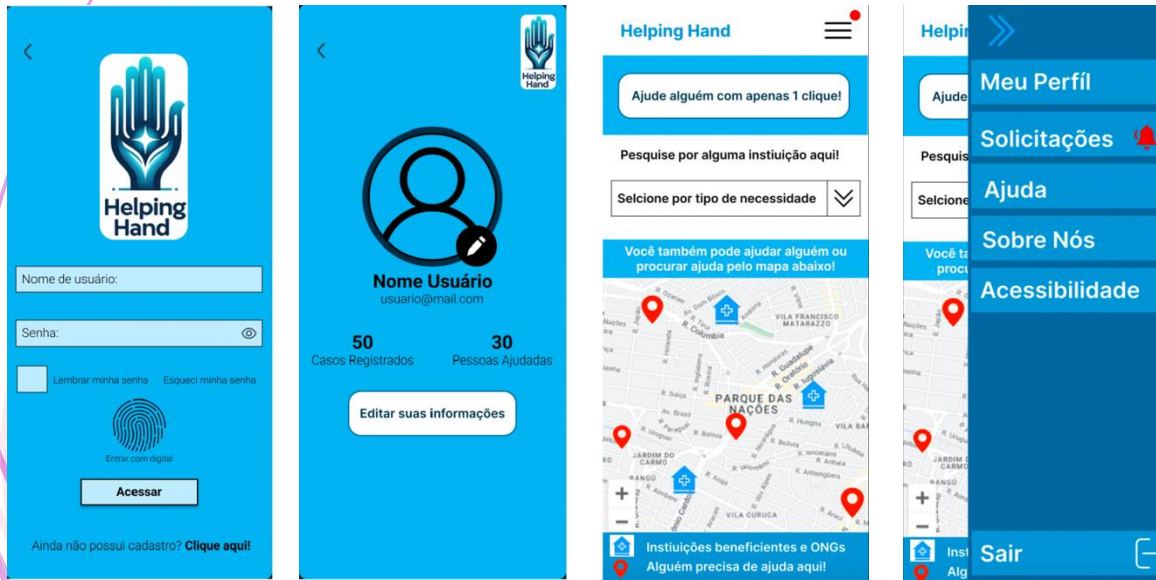
Trata-se de realizar uma encenação representando a situação que se deseja atender com o produto ou processo.

O perfil do usuário é analisado segundo as entrevistas de comportamento e a equipe pode representar o comportamento do consumidor com o produto.

Prototipação

DIGITAL

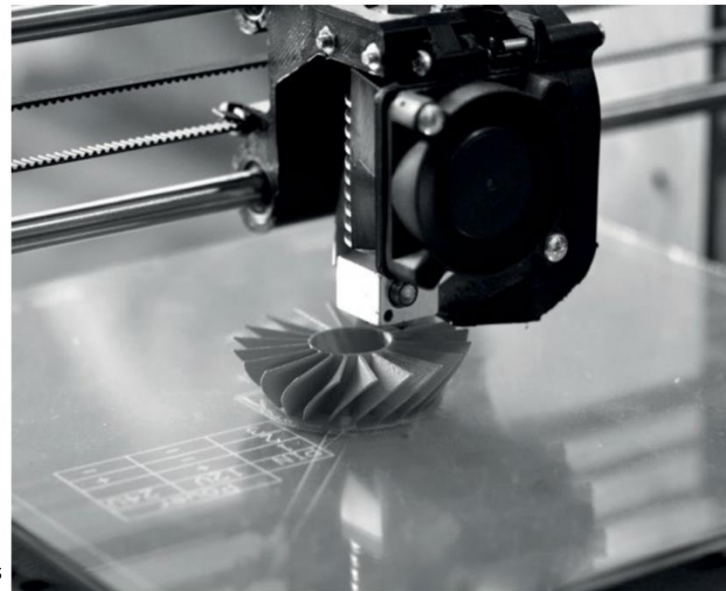
Um dos carros-chefe de prototipação na área de TI na qual são criados recursos digitais (slides, imagens, páginas, protótipos funcionais) que representam o estima-se que o produto ou processo final vai parecer.



PROTOTIPAGEM RÁPIDA

Se refere ao método de produção de protótipos por meio de sistemas aditivos. Isso significa que são adicionados materiais a cada camada, permitindo que protótipos ou modelos sejam produzidos em três dimensões.

Exemplo: impressão 3D.



MarinaGrigorivna/Shutterstock

Em resumo, não importa a técnica que você vai utilizar, o importante mesmo é você gerar protótipos que permitam ao seu cliente entender que tipo de projeto de software você vai oferecer quando concluir seu trabalho.

Com essa etapa nós geramos o famoso **MVP** ou Mínimo Produto Viável. É uma representação do que se espera atingir.

Uma das grandes vantagens do MVP é o baixo custo e redução do prazo de desenvolvimento, pois a solução já foi bem explorada e refinada antes de efetivamente ir para o "forno".

Veja agora alguns exemplos de produtos de sucesso que iniciaram com o MVP.

AMAZON

Iniciou seus serviços como uma livraria convencional, mas, para fins de teste, criou um site muito simplificado e vendeu livros com preços baixos. Logo se desenvolveu e hoje é um dos maiores *marketplaces* do mundo.

FACEBOOK

Quando Mark Zuckerberg lançou o facebook as redes sociais já existiam. Seu MVP foi para conectar os alunos de Harvard. O que fez sucesso foi a simplicidade do APP, tornando-o mundial.

INSTAGRAM

Um MVP que surgiu a partir da necessidade de unir edição de imagens com filtros e postagem na rede. Atualmente é uma das maiores redes sociais do mundo.

SPOTIFY

Iniciou como um simples *streaming* de música. Quando caiu no gosto do público começou a receber novas funcionalidades.

Ferramentas de Prototipação

Existem vários softwares disponíveis para criar os protótipos para um sistema. Cada um deles tem suas características, prós e contras, licenciados ou gratuitos.

Alguns exemplos para conhecimento:



Visio



balsamiq Wireframes



Canva



Figma

** Protótipo estático e Protótipo funcional

Dois conceitos muito importantes que temos que levar em consideração também na prototipação são **User Interface (UI)** e **User Experience (UX)**. Mas, qual a diferença entre os dois?

UX

Está relacionado a como o usuário vai se sentir usando aquele sistema. As telas são fáceis de navegar? É possível encontrar as informações desejadas?

UI

Está relacionado com a parte visual do sistema. Cores mais indicadas para botões de ação, posicionamento de elementos na tela.

Design Thinking

O *Design Thinking* é conhecido como pensamento estratégico do *design* e foi popularizado pelo escritório de *design* IDEO.

Sua forma de abordagem aos problemas é sempre realizada de maneira centrada no ser humano. Em sua essência, o *Design* tem como objetivo primordial a busca pelo bem estar das pessoas.

Segundo Tim Brown, CEO da IDEO, a missão do Design Thinking é "*traduzir observações em insights e, esses insights, em produtos e serviços para melhorar a vida das pessoas*".

Podemos destacar as seguintes vantagens:



Busca de solução de problemas a partir de diversas perspectivas

A solução é encontrada a partir do trabalho colaborativo

Estimula a criatividade

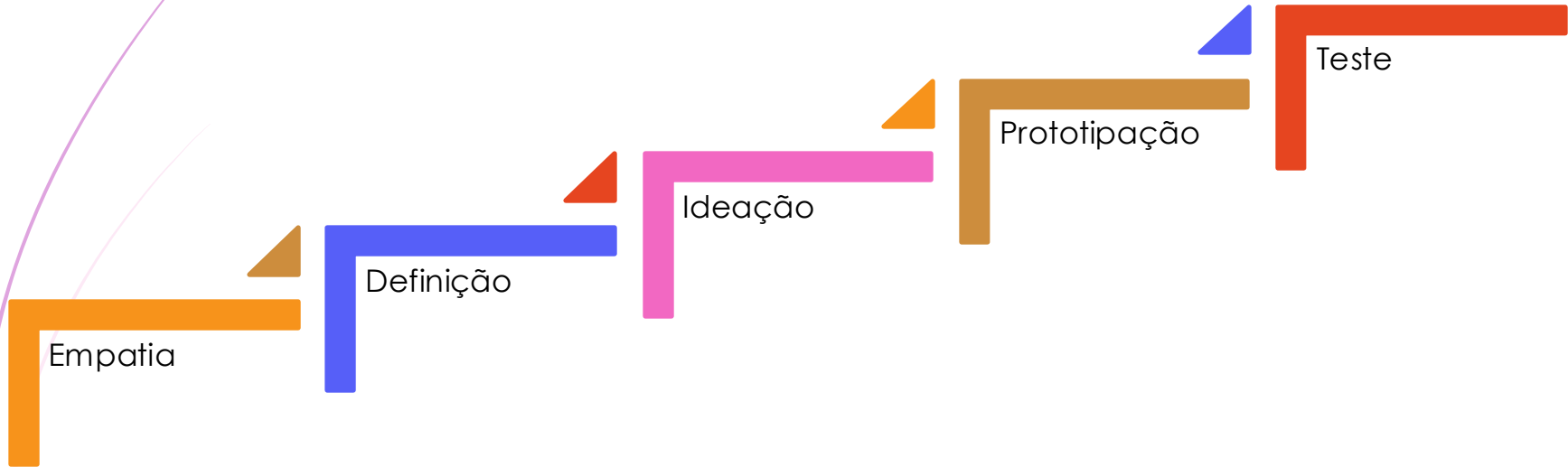
Utiliza diferentes formas de pensar em cenários futuros

Fortalece a capacidade de síntese

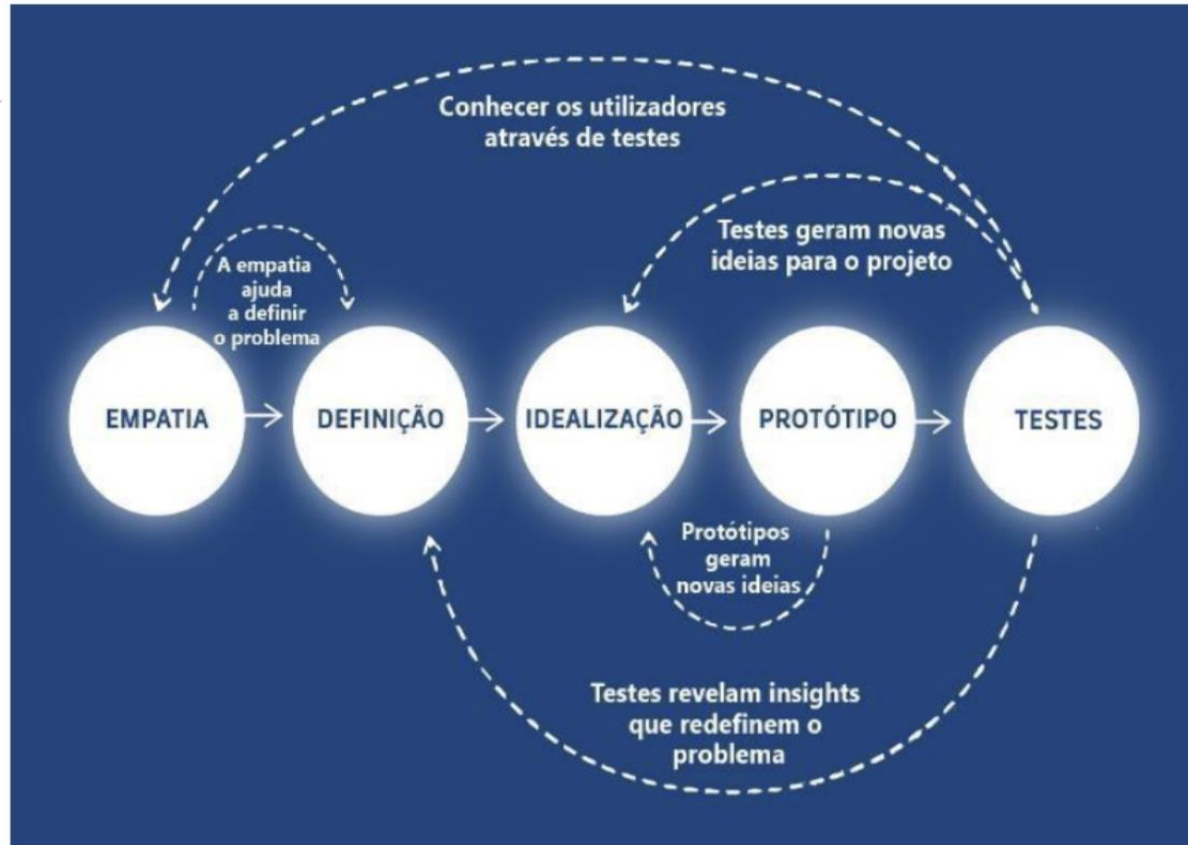
Favorece a inteligência coletiva

Etapas do Design Thinking

As etapas do processo de *Design Thinking*, no geral, são:



Etapas do Design Thinking



Etapas do Design Thinking



EMPATIA

Na etapa da Empatia, ocorre a compreensão do problema e a equipe deve se colocar no lugar do cliente.

Em geral, as equipes são constituídas por:

- *Designer Thinker* – líder do projeto;
- *Design Thinkers* – outros participantes do grupo;
- *Stakeholders* – partes interessadas.

Etapas do Design Thinking



DEFINIÇÃO

Na etapa da Definição, os Design Thinkers devem começar a refinar o problema entendido e começar a projetar soluções eficazes.

Por enquanto, não há regras ou limitações, as ideias podem ser dadas de forma livre, sem filtros, amarras ou limites.

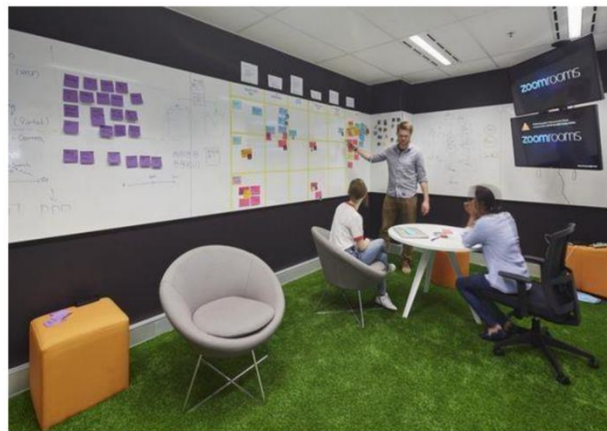
Etapas do Design Thinking

IDEAÇÃO

Na etapa da Ideação, as melhores ideias para solução do problema dos stakeholders “vão para o papel”.

Aqui é interessante realizar *workshops* e *brainstormings* para que a equipe possa dialogar qual o melhor caminho para efetivar a solução que será eleita.

Veja um exemplo de uma sala de *brainstorm*.



Fonte: Design Thinking e o Legal Design, pág 18, Processo

Etapas do Design Thinking



PROTOTIPAÇÃO

Na etapa da Prototipação, a solução que foi eleita para o problema é transformada em algo tangível, palpável e tridimensional.

Aqui aplica-se o conceito do MVP (Mínimo Produto Viável). No entanto, é importante ressaltar que esse protótipo não é a solução final e definitiva, o objetivo dele é justamente alcançá-la. A partir do MVP que há o refinamento, lapidação da solução através dos feedbacks que ela receberá.

Etapas do Design Thinking

TESTE

Na etapa de Teste é quando o protótipo é testado e validado. Por isso, se for possível que os *stakeholders* "pilitem" esse protótipo de alguma forma, é recomendado, pois eles terão uma visão mais real de como será a solução do problema.



ATIVIDADE

Inicie a prototipação das telas do seu projeto. Você e sua equipe devem escolher uma ferramenta gratuita para prototipar.

Cada integrante deve prototipar, no mínimo, **1** tela para apresentação.

4

Engenharia de Requisitos

Descrição de serviços e restrições

Engenharia de Requisitos

Os requisitos de um sistema são as descrições dos serviços que o sistema deve prestar e as restrições a sua operação.

O processo de descoberta, análise, documentação e conferência desses serviços e restrições é chamado de Engenharia de Requisitos.



O autor Alan Davis faz uma definição bem bacana sobre requisitos em seu livro "Software Requirements" de 1993.

"Se uma empresa deseja assinar um contrato para um grande projeto de desenvolvimento de software, ela deve definir suas necessidades de uma maneira suficientemente abstrata para que não haja uma solução predefinida. Os requisitos devem ser escritos de modo que vários concorrentes possam disputar o contrato oferecendo, talvez, maneiras diferentes de satisfazer as necessidades da empresa cliente. Depois de assinado o contrato, o contratado deve escrever uma definição mais detalhada do sistema para o cliente, de modo que ele entenda e valide o que o software fará. Esses dois documentos podem ser reunidos em um documento de requisitos do sistema."

Requisitos de Usuário

- São declarações feitas em linguagem natural e representadas em diagramas, dos serviços que se espera que o sistema forneça e suas limitações.

Requisitos de Sistema

- São descrições detalhadas das funções, dos serviços e das restrições do software. O documento de requisitos do sistema, também chamado de especificação funcional, deve definir exatamente o que deve ser implementado.

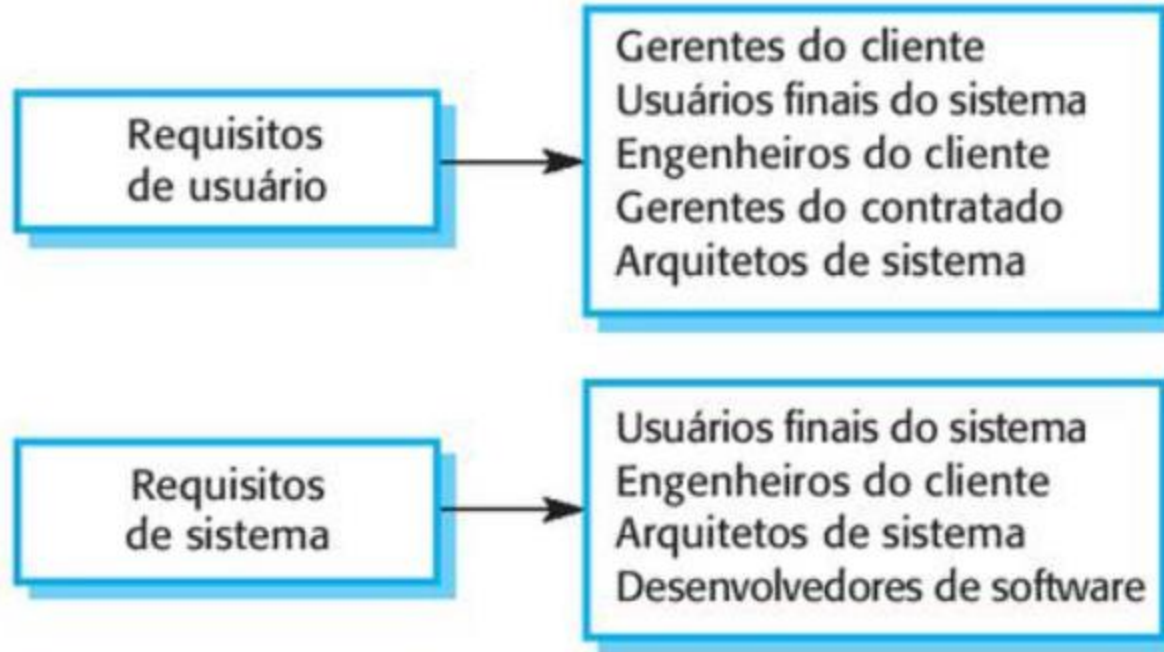
Definição de requisitos de usuário

- 1.** O sistema Mentcare deve gerar relatórios de gestão mensais, mostrando o custo dos medicamentos prescritos por cada clínica naquele mês.

Especificação dos requisitos de sistema

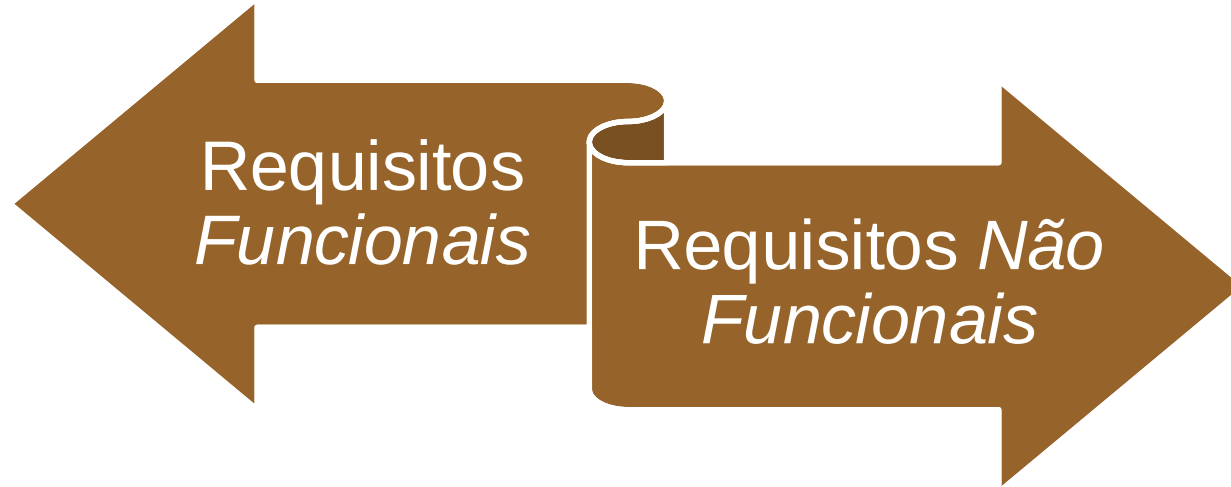
- 1.1** No último dia útil de cada mês, deve ser gerado um resumo dos medicamentos prescritos, seu custo e a clínica que os prescreveu.
- 1.2** O sistema deve gerar o relatório para impressão após as 17h30 do último dia útil do mês.
- 1.3** Deve ser criado um relatório para cada clínica, listando o nome de cada medicamento, a quantidade total de prescrições, a quantidade de doses prescritas e o custo total dos medicamentos prescritos.
- 1.4** Se os medicamentos estiverem disponíveis em dosagens diferentes (por exemplo, 10 mg, 20 mg etc.) devem ser criados relatórios diferentes para cada dosagem.
- 1.5** O acesso aos relatórios de medicamentos deve ser restrito aos usuários autorizados, conforme uma lista de controle de acesso produzida pela gestão.

Engenharia de Requisitos



Engenharia de Requisitos

Pensando em Requisitos de Sistema, podemos separá-los em 2 outras categorias:



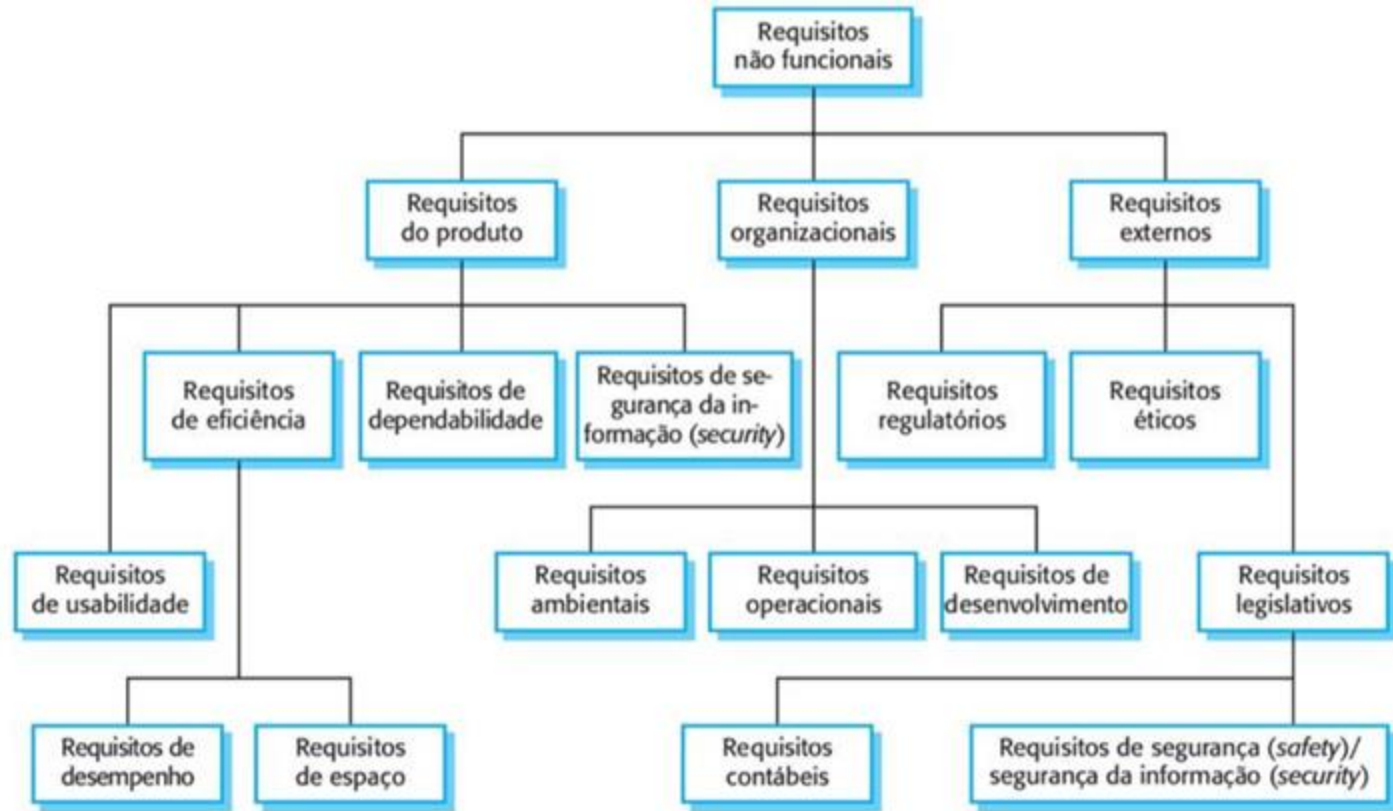
REQUISITOS FUNCIONAIS descrevem:

- ▶ Os serviços que o sistema deve fornecer;
- ▶ Como ele deve reagir a determinadas entradas;
- ▶ Como deve se comportar em determinadas situações;
- ▶ E, se cabível, o que o sistema não deve fazer.

REQUISITOS NÃO FUNCIONAIS descrevem:

- ▶ Restrições dos serviços e/ou funções do sistema;
- ▶ Restrições de tempo;
- ▶ Restrições de desenvolvimento;
- ▶ Restrições impostas por padrões.

Engenharia de Requisitos



Notações utilizadas para escrever requisitos

Notação	Descrição
Sentenças em linguagem natural	Os requisitos são escritos usando frases numeradas em linguagem natural. Cada frase deve expressar um requisito.
Linguagem natural estruturada	Os requisitos são escritos em linguagem natural em um formulário ou <i>template</i> . Cada campo fornece informações sobre um aspecto do requisito.
Notações gráficas	Modelos gráficos, suplementados por anotações em texto, são utilizados para definir os requisitos funcionais do sistema. São utilizados com frequência os diagramas de casos de uso e de sequência da UML.
Especificações matemáticas	Essas notações se baseiam em conceitos matemáticos como as máquinas de estados finitos ou conjuntos. Embora essas especificações inequívocas possam reduzir a ambiguidade em um documento de requisitos, a maioria dos clientes não compreende uma especificação formal. Eles não conseguem averiguar se ela representa o que desejam e relutam em aceitar essa especificação como um contrato do sistema (discutirei essa a

Capítulo 10, que



5

Modelagem de Sistemas

UML – Unified Modeling Language

Modelagem de Sistemas

Modelagem de sistemas significa, basicamente, representar um sistema usando algum tipo de notação gráfica baseada nos tipos de diagrama em UML (*Unified Modeling Language* | Linguagem de Modelagem Unificada).

Os modelos são usados durante o processo de engenharia de requisitos, para ajudar a derivar os requisitos detalhados de um sistema.

A UML surgiu de um trabalho nos anos 1990 sobre modelagem orientada a objetos e contém um conjunto de 13 tipos diferentes de diagramas que podem ser usados para modelar sistemas de software. No entanto, um levantamento feito em 2007 concluiu que a maioria dos usuários da UML entendiam que 5 desses tipos já poderiam representar os fundamentos de um sistema.

UML – Tipos de Digrama

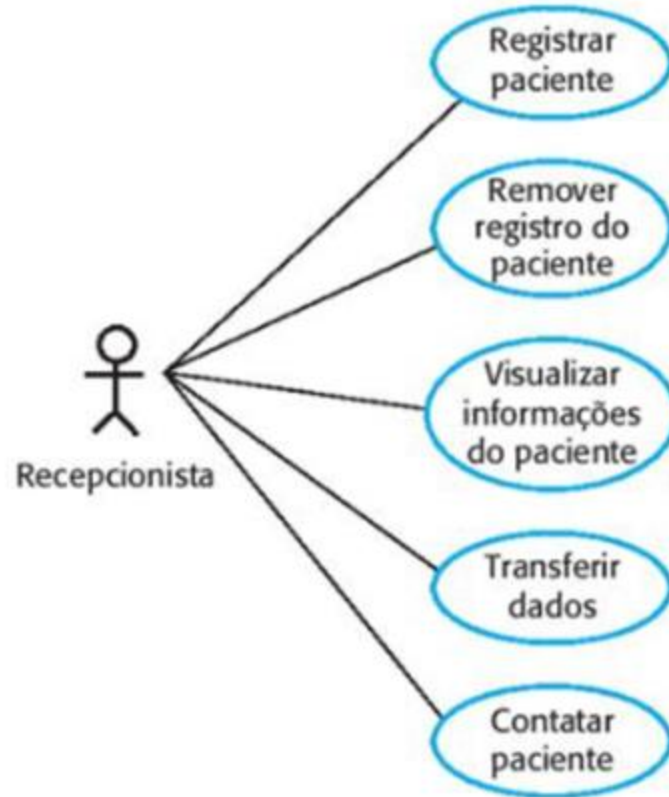
- ▶ **Diagramas de Atividades:** mostram as atividades envolvidas em um processo ou processamento de dados;
- ▶ **Diagramas de Caso de Uso:** mostram as interações entre um sistema e seu ambiente;
- ▶ **Diagramas de Sequência:** mostram as interações entre os atores e o sistema e entre os componentes do sistema;
- ▶ **Diagramas de Classes:** mostram as classes de objetos no sistema e as associações entre elas;
- ▶ **Diagramas de máquinas de estados:** mostram como o sistema reage a eventos internos e externos.

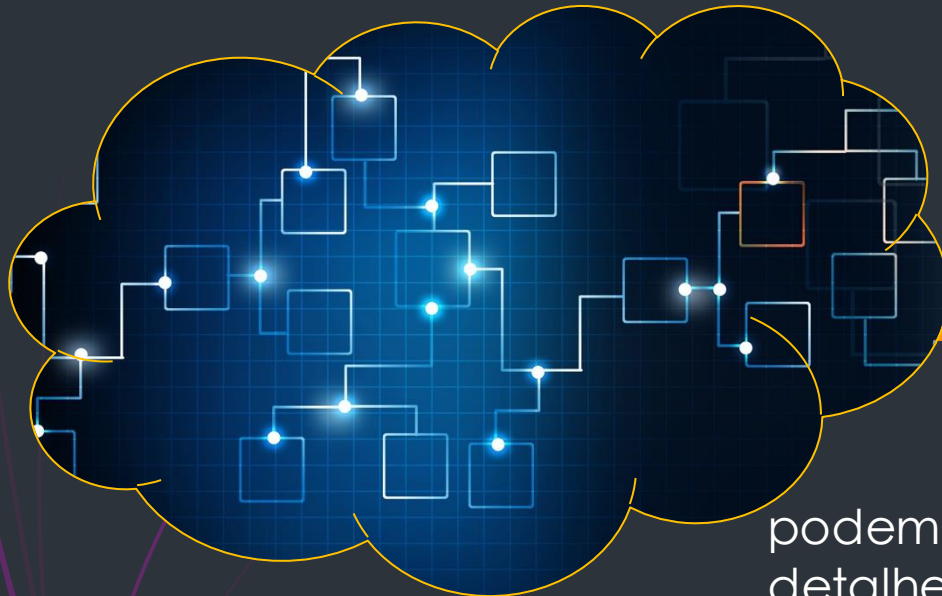
UML – Diagrama de Caso de Uso



Fluxo do processo

UML – Diagrama de Caso de Uso





ATIVIDADE

Escolha um dos diagramas que podem ser feitos com UML, estude mais detalhes sobre ele, faça o diagrama e apresente para seus colegas!

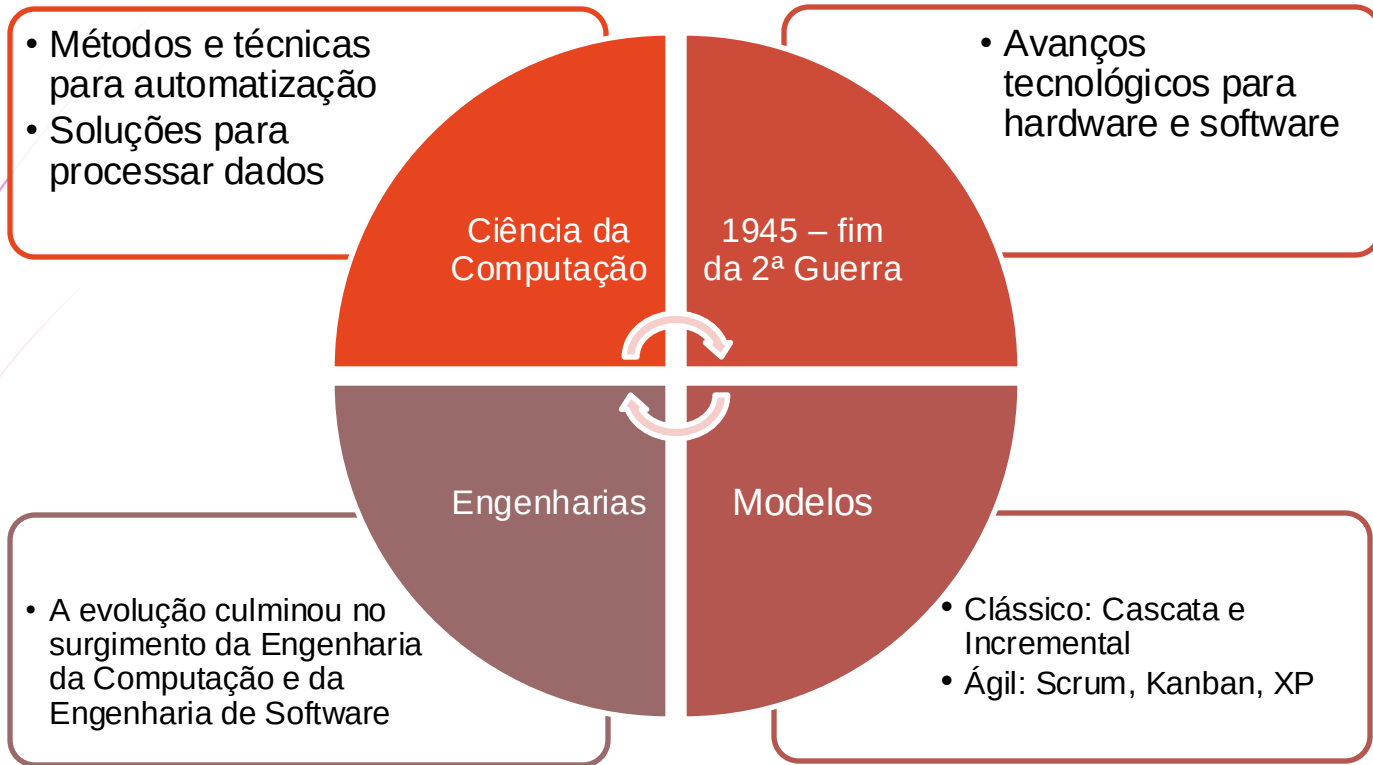
Obs: use o tema do seu grupo para diagramar.



6

Metodologia de Desenvolvimento Ágil

Metodologia de Desenvolvimento Ágil



Manifesto Ágil

Em 2001, foi realizado um estudo pelo *The Standish Group International* que revelou os seguintes pontos:

- ▶ Em média, os atrasos representam 63% mais tempo do que o estimado;
- ▶ Os projetos que não cumpriram o orçamento custaram, em média, 45% a mais;
- ▶ No geral, apenas 67% das funcionalidades prometidas foram efetivamente entregues.

Esse estudo foi nominado e publicado com CHAOS Report e foram considerados mais de 280 mil projetos de software nos EUA. Nessa ocasião, foi identificada uma crise de software.

Frente a isso, 17 engenheiros de software se reuniram para pensar o que poderia ser feito. A partir daí surgiu o manifesto ágil.

Manifesto Ágil

Estamos descobrindo maneiras melhores de desenvolver software, fazendo-o nós mesmos e ajudando outros a fazerem o mesmo. Através deste trabalho, passamos a valorizar:

Indivíduos e interações *mais que processos e ferramentas*
Software em funcionamento *mais que documentação abrangente*
Colaboração com o cliente *mais que negociação de contratos*
Responder a mudanças *mais que seguir um plano*

Ou seja, mesmo havendo valor nos itens à direita, valorizamos mais os itens à esquerda.

Métodos Ágeis

Todos os métodos ágeis sugerem que o software deve ser desenvolvido e entregue incrementalmente.

Eles se baseiam em diferentes processos inspirados pelo manifesto.

A comunicação é sempre contínua.

Os times podem utilizar a comunicação informal.

Sistemas de pequeno e médio porte.

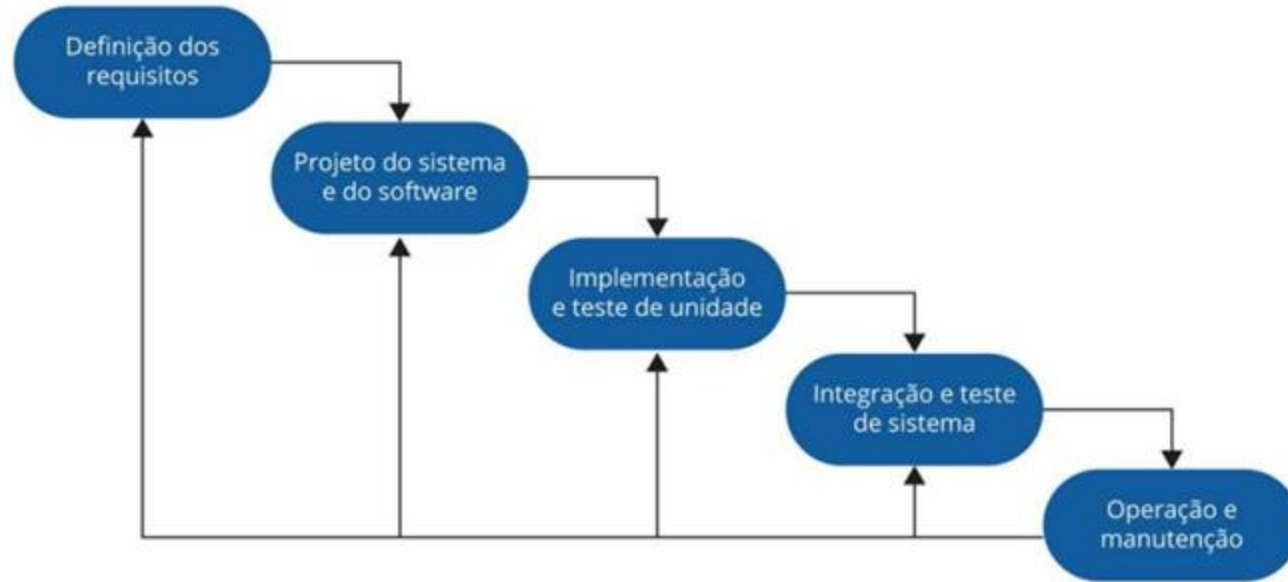
Sistemas personalizados.

Princípios dos métodos ágeis

Princípio	Descrição
Envolvimento do cliente	Os clientes devem ser envolvidos em todo o processo de desenvolvimento. Seu papel é fornecer e priorizar novos requisitos de sistema e avaliar as iterações do sistema.
Acolher as mudanças	Tenha em mente que os requisitos do sistema mudam e, portanto, deve-se projetar o sistema para acomodar essas mudanças.
Entrega incremental	O software é desenvolvido em incrementos, e o cliente especifica os requisitos incluídos em cada um deles.
Manter a simplicidade	Deve-se ter como foco a simplicidade, tanto do software que está sendo desenvolvido quanto do processo de desenvolvimento. Sempre que possível, trabalhe ativamente para eliminar a complexidade do sistema.
Pessoas, não processos	As habilidades do time de desenvolvimento devem ser reconhecidas e aproveitadas da melhor maneira possível. Seus membros devem ter liberdade para desenvolver seu modo próprio de trabalhar sem se prender a processos determinados.

Modelo Cascata x Modelo Ágil

Modelo Cascata



Modelo Cascata x Modelo Ágil

Modelo Cascata

Esse modelo traz como benefício o foco no processo e execução por meio da especialização das atividades, ou seja, cada pessoa atua numa etapa e faz o seu melhor. Quando um trabalho termina, começa o de outra pessoa na etapa em sequência.

Sommerville orienta que esse modelo é adequado para sistemas embarcados, sistemas críticos e grandes sistemas de software. E, não é recomendado para situações em que a comunicação informal do time é possível e nas quais os requisitos de software mudam rapidamente.

Modelo Cascata x Modelo Ágil

Modelo Incremental

O modelo em cascata gerava alguns problemas de lentidão e burocracia, então foi criado um segundo modelo que busca entregas menores e incrementais. Desta forma, há uma implementação inicial, obtém-se o feedback dos usuários ou terceiros e o software vai evoluindo através de várias versões até alcançar todas as funcionalidades necessárias.

Esse modelo é a base para todas as abordagens ágeis.

Modelo Cascata x Modelo Ágil

	TRADICIONAL	ÁGIL
<i>Pressupostos fundamentais</i>	Sistemas totalmente especificáveis, previsíveis; desenvolvidos a partir de um planejamento extensivo e meticuloso	Software adaptativo e de alta qualidade; pode ser desenvolvido por equipes pequenas utilizando os princípios da melhoria contínua do projeto e testes orientados a rápida resposta a mudança.
<i>Controle</i>	Orientado a processos	Orientado a pessoas
<i>Estilo de gerenciamento</i>	Comandar e controlar	Liderar e colaborar
<i>Gestão do conhecimento</i>	Explícito	Tácito
<i>Atribuição de papéis</i>	Individual – favorece a especialização	Times auto-organizáveis – favorece a troca de papéis
<i>Comunicação</i>	Formal	Informal
<i>Ciclo do projeto</i>	Guiado por tarefas ou atividades	Guiado por funcionalidades do produto
<i>Modelo de desenvolvimento</i>	Modelo de ciclo de vida (cascata, espiral ou alguma variação)	Modelo iterativo e incremental de entregas
<i>Forma/estrutura</i>	Mecânica (burocrática com muita formalização)	Orgânica (flexível e com incentivos a participação e cooperação social)

eXtreme Programming (XP)

Durante essa época de estudos e descoberta dos métodos ágeis, foi quando surgiu a abordagem mais importante para a mudança da cultura de desenvolvimento de software: a Programação Extrema.

O XP foi criado por Kent Beck, Ward Cunningham e Ron Jeffries, todos signatários do manifesto ágil.

Aqui, são desenvolvidas várias versões do sistema, por diferentes programadores, e elas são integradas e testadas em um dia.

Seu objetivo principal é buscar a constante satisfação do cliente. Com ele, o time de desenvolvimento deve abraçar as mudanças e aceitá-las a todo momento. Tudo é feito em equipe, gestores, clientes e desenvolvedores são parceiros de igual peso.

Veja como funciona o processo de produção do incremento de um sistema utilizando XP.

eXtreme Programming (XP)



No XP, os requisitos são expressos em cenários (chamados histórias do usuário) e eles são implementados como uma série de tarefas.

Os programadores trabalham em pares e desenvolvem testes para cada tarefa antes de escreverem o código.

O XP prega 4 valores principais e 10 práticas.

eXtreme Programming (XP) – 4 valores



Feedback



Comunicação



Simplicidade



Coragem

eXtreme Programming (XP) – 10 práticas

Propriedade coletiva

- Os pares de desenvolvedores trabalham em todas as áreas do sistema de modo que não se desenvolvem "ilhas de conhecimento" e todos os desenvolvedores assumem a responsabilidade por todo código. Qualquer um pode mudar qualquer coisa.

Integração contínua

- Assim que o trabalho em uma tarefa é concluído, ele é integrado ao sistema completo. Após qualquer integração desse tipo, todos os testes de unidade no sistema devem passar.

Planejamento incremental

- Os requisitos são registrados em "cartões de histórias" e as histórias a serem incluídas em um lançamento são determinadas de acordo com o tempo disponível e com sua prioridade relativa. Os desenvolvedores decompõem essas histórias em "tarefas" de desenvolvimento.

eXtreme Programming (XP) – 10 práticas

Representante do cliente

- Um representante do cliente deve estar disponível em tempo integral para o time de programação. Em um processo como esse, o cliente é um membro do time de desenvolvimento, sendo responsável por levar os requisitos do sistema ao time, visando sua implementação.

Programação em pares

- Os desenvolvedores trabalham em pares, verificando o trabalho dos outros e prestando apoio para um bom trabalho sempre.

Refatoração

- Todos os desenvolvedores devem refatorar o código continuamente assim que encontrarem melhorias de código. Isso mantém ele simples e manutenível.

eXtreme Programming (XP) – 10 práticas

Projeto (design) simples

- Deve ser feito o suficiente de projeto (design) para satisfazer os requisitos atuais e nada mais.

Lançamentos pequenos

- O mínimo conjunto útil de funcionalidade que agregue valor ao negócio é desenvolvido em primeiro lugar. Os lançamentos do sistema são frequentes e acrescentam funcionalidade à primeira versão de uma maneira incremental.

Ritmo sustentável

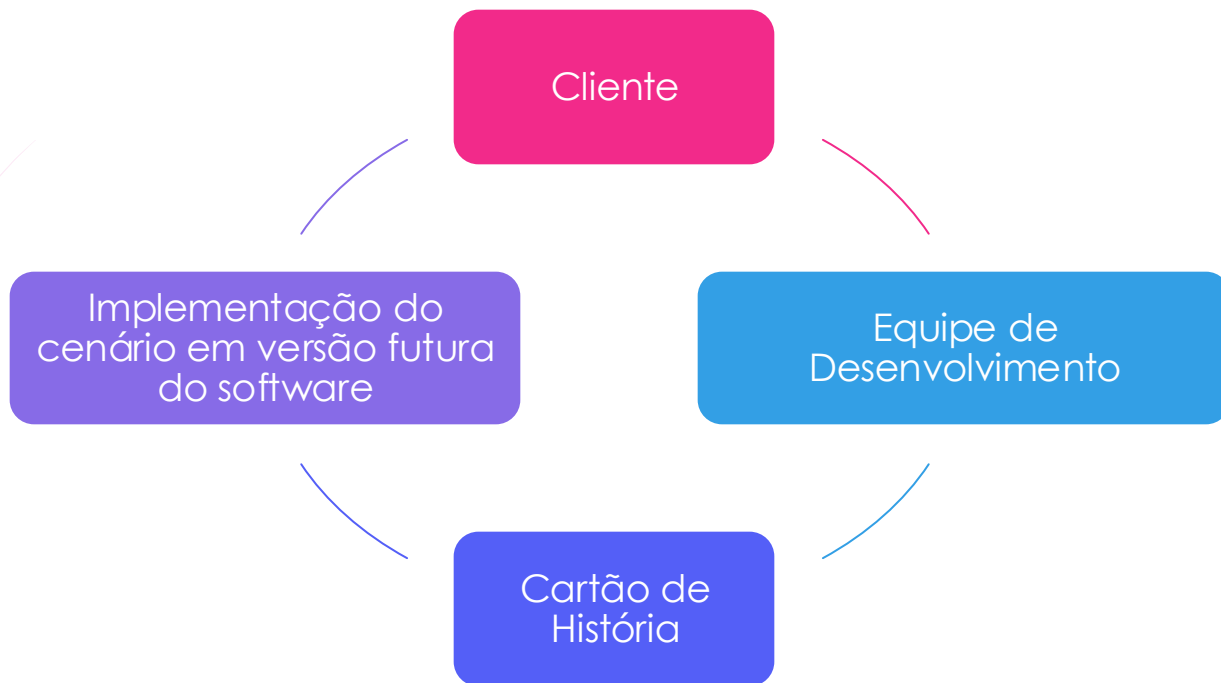
- Grande quantidade de horas extras não é considerado aceitável, já que o efeito líquido muitas vezes é a diminuição da qualidade do código e da produtividade no médio prazo

Desenvolvimento com testes a priori (*test-first*)

- Um framework automatizado de testes de unidade é utilizado para escrever os testes de um novo pedaço de funcionalidade antes que ela própria seja implementada.

eXtreme Programming (XP)

Como os requisitos de software sempre mudam, nos métodos ágeis utiliza-se o recurso de **histórias do usuário** ao invés de ter uma atividade de engenharia de requisitos específica ou independente.



Prescrição de medicação

Kate é uma médica que deseja prescrever medicação para um paciente que frequenta sua clínica. O registro do paciente já é exibido em seu computador, então ela deve clicar no campo de medicação e selecionar 'medicação atual', 'nova medicação' ou 'substâncias'.

Se escolher 'medicação atual', o sistema pedirá para que ela verifique a dose; se quiser mudá-la, Kate fornecerá a nova dose e depois deverá confirmar a prescrição.

Se escolher 'nova medicação', o sistema presumirá que ela sabe qual medicação prescrever. Ao digitar as primeiras letras da medicação, o sistema exibirá uma lista de possíveis medicamentos que começam com essas letras. Ao escolher a medicação necessária, o sistema responderá pedindo que ela confirme se a medicação escolhida está correta. Ela deverá fornecer a dose e depois confirmar a prescrição.

Se ela escolher 'substâncias', o sistema exibirá uma caixa de busca para a substância aprovada, e então ela poderá pesquisar o medicamento que deseja. Feita a busca, lhe será solicitado que confirme se o medicamento selecionado está correto. Ela deverá fornecer a dose e depois confirmar a prescrição.

O sistema sempre verificará se a dose está dentro da faixa aprovada. Se não estiver, será pedido a Kate que faça uma alteração.

Depois de confirmar a prescrição, ela será exibida para conferência. Kate deverá clicar em 'OK' ou em 'Alterar'. Se 'OK' for selecionado, a prescrição será registrada no banco de dados de auditoria. Se clicar em 'Alterar', o processo de 'Prescrição de medicação' é executado novamente.

eXtreme Programming (XP)

Depois de escritas, as histórias do usuário viram tarefas para o time de desenvolvimento, que estima o esforço e os recursos necessários para realiza-las.

Tarefa 1: Mudar a dose do medicamento prescrito

Tarefa 2: Seleção de substância

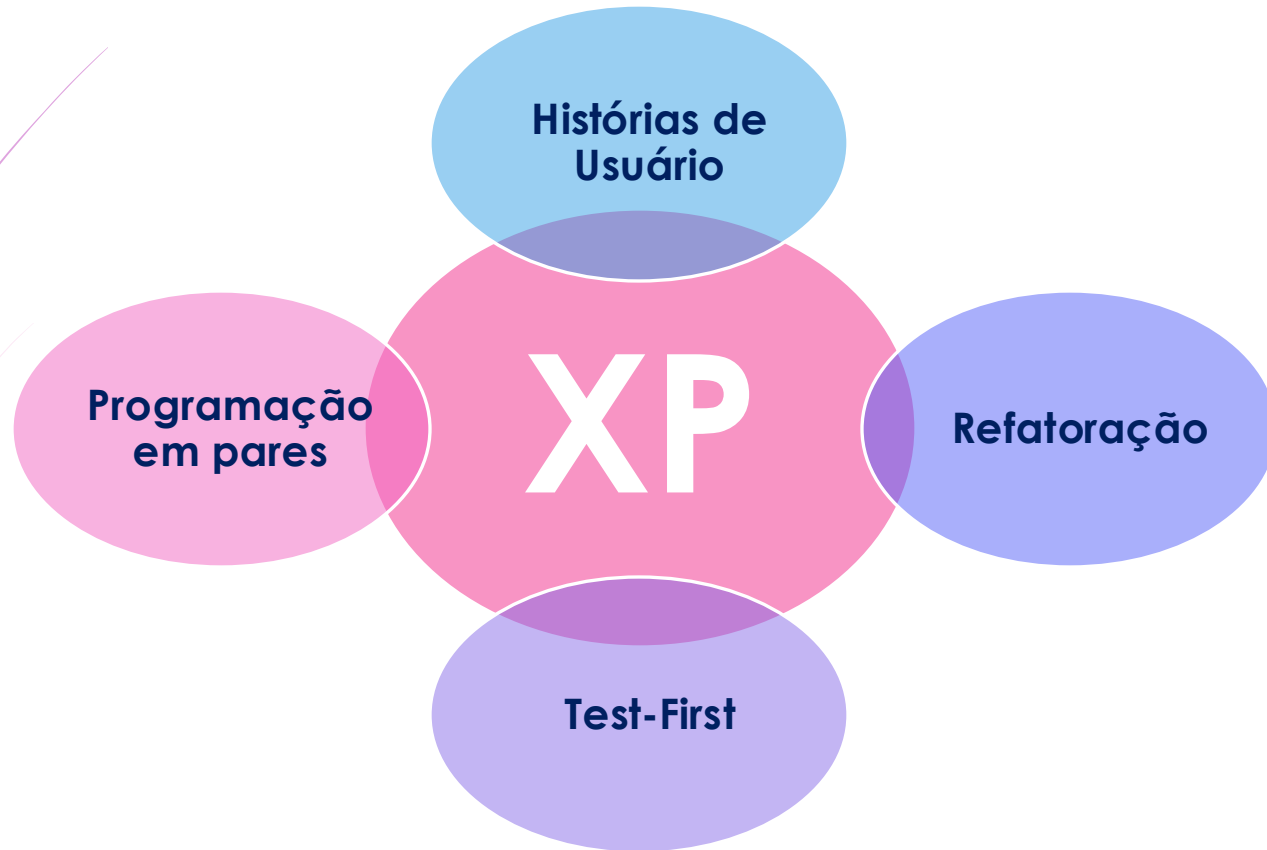
Tarefa 3: Verificação da dose

A verificação da dose é uma precaução de segurança para conferir se o médico não prescreveu uma dose perigosamente pequena ou grande.

Usando a identificação da substância para o nome genérico do medicamento, procure a substância e recupere a dose máxima e a mínima recomendadas.

Confira a dose prescrita em relação ao máximo e ao mínimo. Se estiver fora da faixa, emita uma mensagem de erro dizendo que a dose é alta ou baixa demais. Se estiver dentro da faixa, habilite o botão "Confirmar".

eXtreme Programming (XP)



eXtreme Programming (XP)

Exemplo de um caso de teste:

Teste 4: Conferir a dose

Entrada:

1. Um número em mg representando uma única dose de medicamento.
2. Um número representando a quantidade de doses únicas por dia.

Testes:

1. Teste das entradas em que a dose única está correta, mas a frequência é alta demais.
2. Teste das entradas em que a dose única é alta demais e baixa demais.
3. Teste das entradas em que a dose única * frequência é alta demais e baixa demais.
4. Teste das entradas em que a dose única * frequência está no intervalo permitido.

Saída:

'OK' ou mensagem de erro indicando que a dose está fora do intervalo seguro.

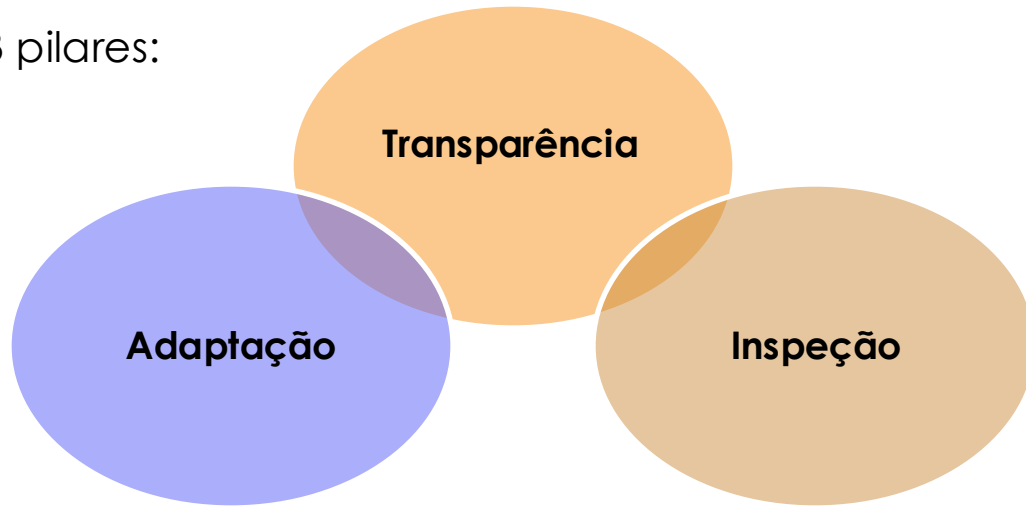
Gerenciamento ágil de projetos



O SCRUM é um framework ágil, criado por Ken Schwaber e Jeff Sutherland. A ideia foi baseada na formação dos times de rugby onde cada jogador do mesmo time tem uma especialidade e, juntos, conseguem resultados melhores.

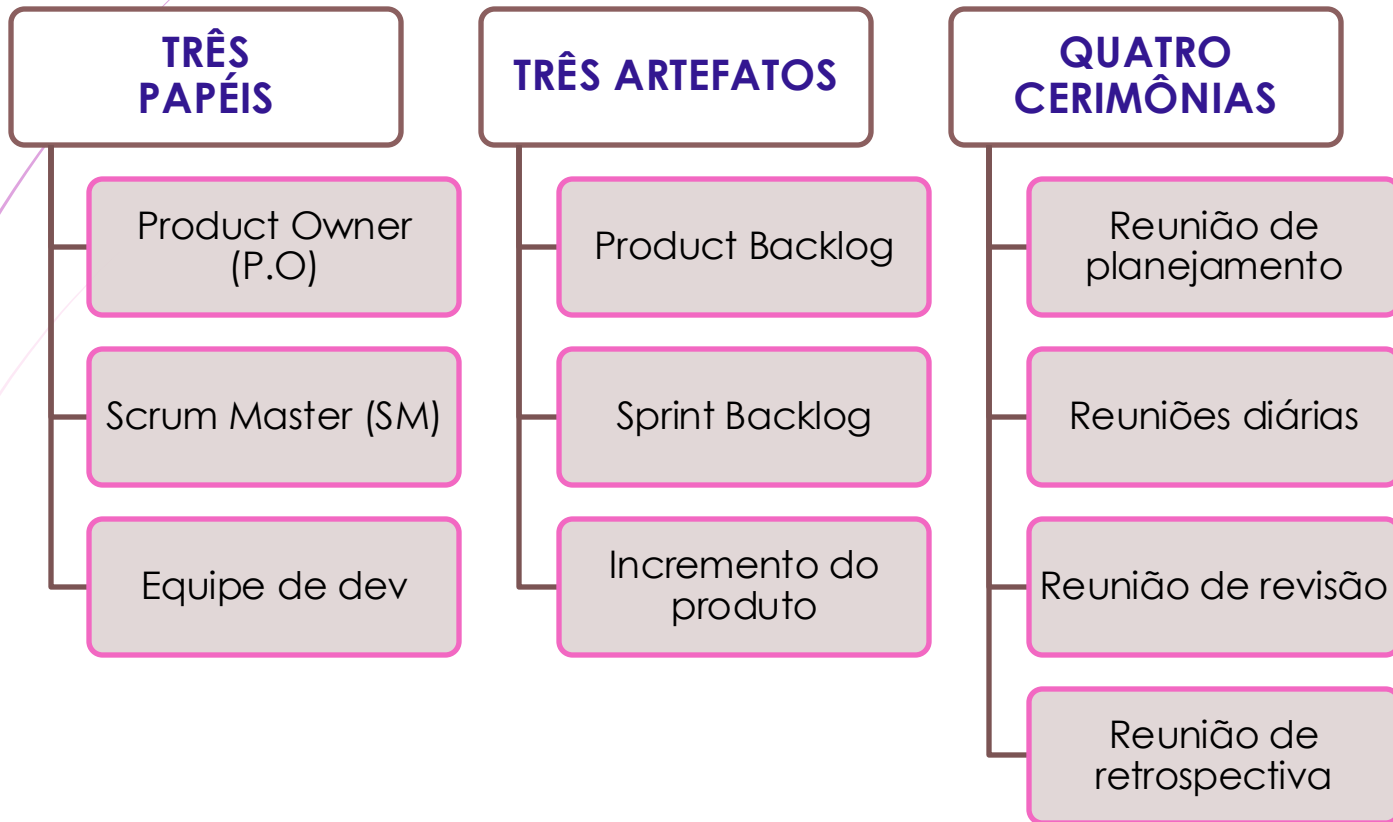
Em seu fundamento, o conhecimento vem da experiência e da tomada de decisões com base no que é conhecido.

Possui 3 pilares:



SCRUM

No framework SCRUM temos:



SCRUM – 3 pápeis



PO | Product Owner | Dono do Produto

- ▶ É o responsável por maximizar o valor do produto resultado do trabalho do time de desenvolvimento.



SM | Scrum Master

- ▶ É o responsável por promover, suportar e gerenciar as práticas do Scrum na equipe conforme definido no Guia do Scrum.



Equipe de Desenvolvimento

- ▶ São os profissionais que realizam o trabalho de produção, ou seja, que transformam as definições do projeto em um produto final utilizável.

SCRUM - Sprint

Sprint é um ciclo completo de desenvolvimento de sistema com duração fixa de tempo.

*** De 2 a 4 semanas*

SCRUM – 3 Artefatos



Product Backlog | Backlog do produto

- ▶ É uma lista ordenada de funcionalidades que precisam ser desenvolvidas.



Sprint Backlog / Backlog do Sprint

- ▶ É o conjunto de itens que foram selecionados para serem implementados durante o *sprint*.



Incremento do produto

- ▶ É o resultado final de cada *sprint*.

SCRUM – 4 cerimônias



- ▶ **Reunião de planejamento:** a equipe alinha e planeja o que será feito naquela *sprint*;
- ▶ **Reuniões diárias:** a equipe alinha o que foi feito no dia anterior e o que será feito no próprio dia;
- ▶ **Reunião de revisão:** a equipe alinha todos os itens do *sprint* antes da entrega;
- ▶ **Reunião de retrospectiva:** a equipe alinha como foi o andamento da última *sprint*, o que pode ser melhorado, o que precisa ser mudado e o que deve ser mantido.

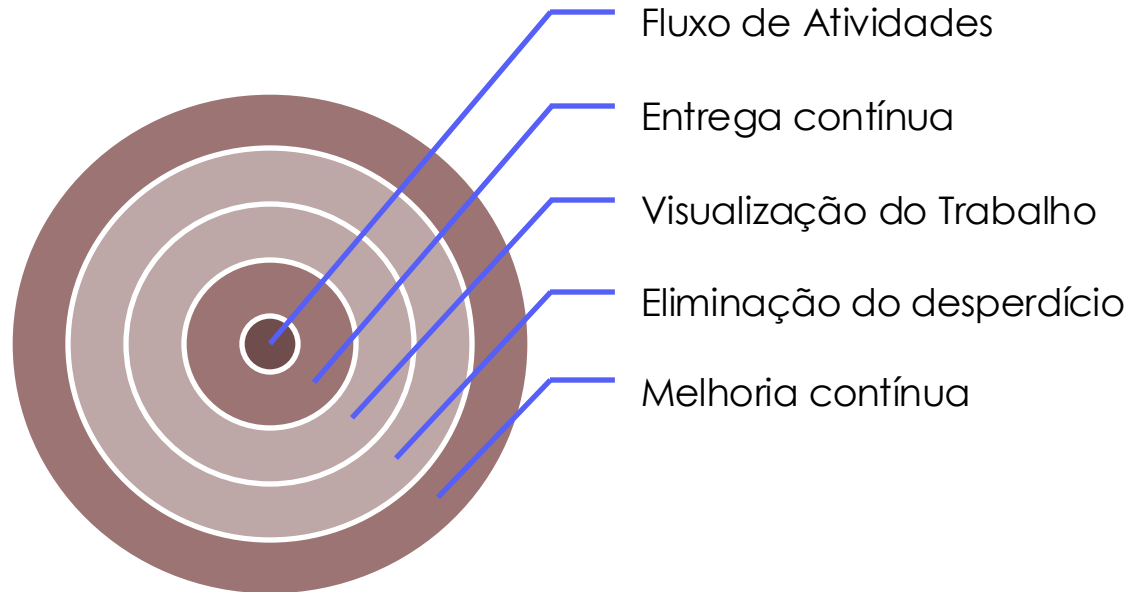
Analizando os vários casos de sucesso na aplicação desta metodologia ágil, conclui-se que o que os usuários mais gostam são:

- O produto é decomposto em um conjunto de “pedaços” gerenciáveis e compreensíveis aos quais os *stakeholders* podem se referir.
- Requisitos instáveis não impedem o progresso.
- O time inteiro tem visibilidade de tudo e, conseqüentemente, a comunicação e disposição de seus membros são melhores.
- Os clientes veem a entrega dos incrementos na hora e obtém feedback de como o produto funciona.
- A confiança entre clientes e desenvolvedores é estabelecida, criando uma cultura positiva na qual todos esperam que o projeto tenha sucesso.

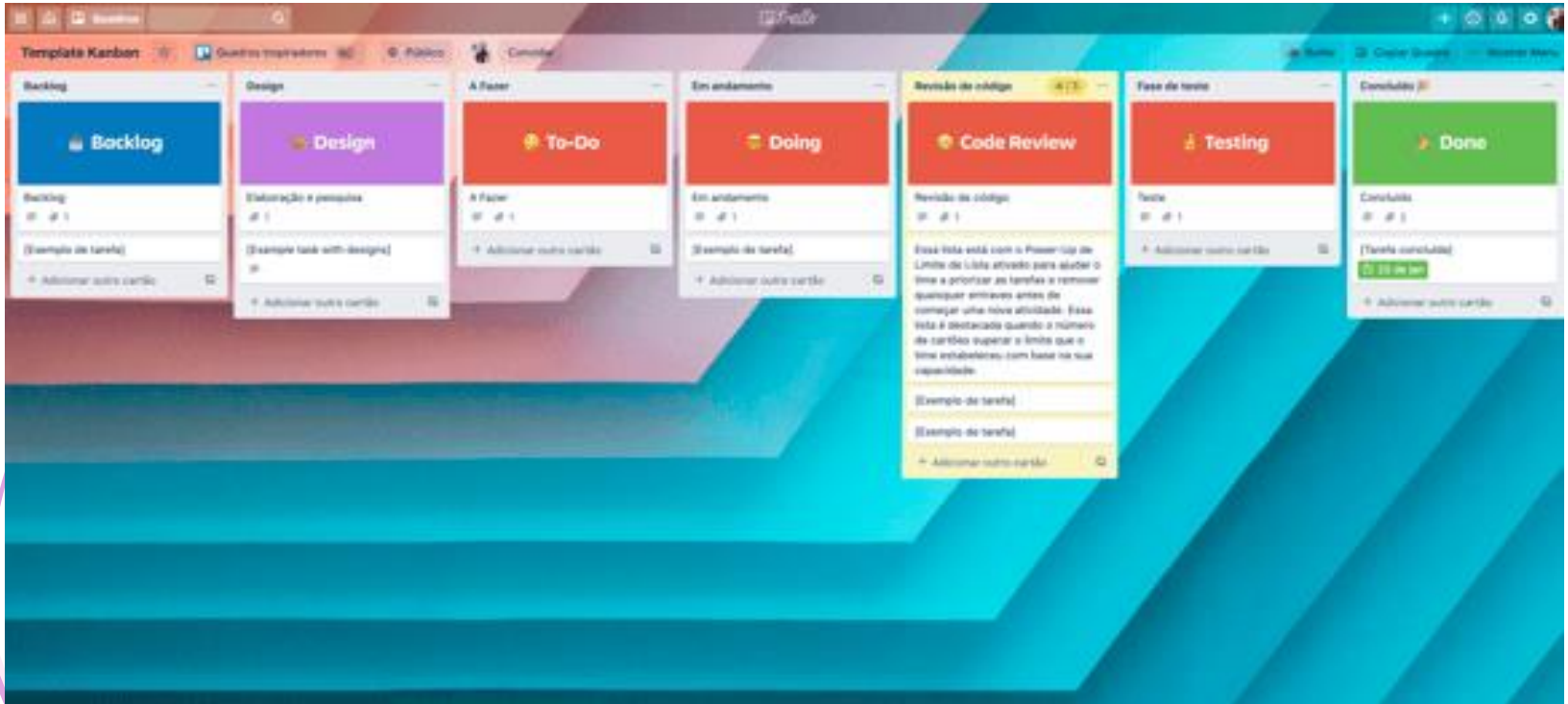
Método KANBAN

O método Kanban foi criado por David Anderson e é inspirado no modelo Toyota de produção (kanban – repare no “k” minúsculo).

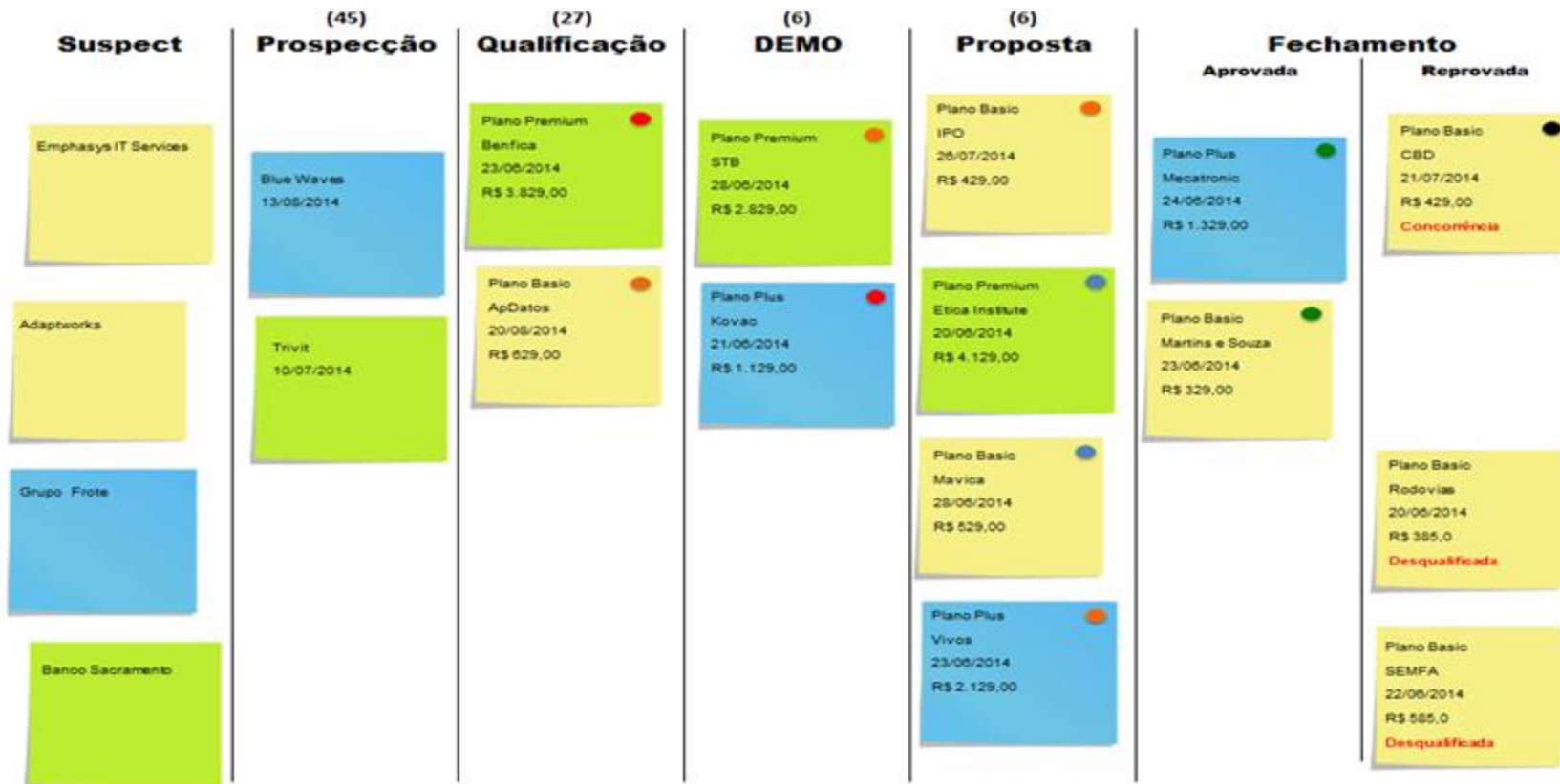
Kanban valoriza:



Método KANBAN



Método KABAN



Método KANBAN

- ▶ São utilizados *cards* (cartões) para representar as tarefas que devem ser desempenhadas;
- ▶ Esse tipo de disposição ajuda na identificação de gargalos que são acúmulos (excessos) de trabalho em alguma etapa. Quando identificado, o time deverá se reorganizar a fim de eliminar a sobrecarga;
- ▶ O Kanban trabalha com a visão de trabalho puxado, e não empurrado como no Scrum, por exemplo.
 - ▶ Os *cards* já começados são considerados WIP = *work in progress*;
 - ▶ À medida que um item é finalizado, o WIP diminui e um espaço de trabalho é liberado. Com isso, uma nova tarefa já pode ser iniciada, caracterizando um sistema puxado de trabalho.
 - ▶ A limitação de WIP, além de evitar os gargalos, faz com que a entrada de novas tarefas fique baseada na disponibilidade de espaços de trabalho e não baseada na capacidade de trabalho do *sprint*.

Método Kanban + Scrum – “Scrumban”

	TO DO					
STORIES	BACKLOG	THIS SPRINT	NEXT	DOING 3 / 2	FOR APPROVAL	SPRINT DONE
+ add task	+ add task	+ add task	+ add task	+ add task	+ add task	+ add task
prepare the invoices	Rework the main page layout	[high]	prepare invoice 569	task 34 C	promo codes M Team	prepare the invoice 4758
prepare the invoice 45	prepare the invoices	task 113	prepare the invoice 4758	promo codes M Team	task 34 B	use case 862 re-do
prepare the presentation for WCY	prepare the invoice 569	give away the marketing info to DC	prepare a speech for the presentation in DC on December 30 2016	use case 862 re-do	as a user I can edit all of my details	use case 56 : check for updates
get all archived documents for the tax office	prepare the invoices	prepare the invoice 667	promo codes MTeam		prepare the invoice 223	use case 56
prepare the invoices for ZDBank	Task 1	redo the standign orders for 2016	prepare the invoice 223		Contact Marleha	re
use case 56	promo codes MTeam	hire an office assistant	34 scenario		as a user I can edit all of my details	QAS #61
german text bug fix	scan the server for malware		test the Cycle Time script		test	set the domain name to new
	use case 56		prepare the invoice 4758		go go go	get the documents for the tax office
	as a user I can edit all of my details		give away the marketing info to DC		as a user I can edit all of my details in Prom View	check order 88
	give away the marketing info to DC					give away the marketing info to DC
	use case 56					use case 862 re-do
						set the subdomain name to sub...



7

Teste de Software

Definição, tipos e utilização

Os Testes de Software tem o objetivo de provar que um programa faz o que foi planejado para fazer e também para descobrir defeitos antes que ele seja colocado para uso real/oficial.

Para testar um programa, são inseridos dados não oficiais nas funcionalidades que foram desenvolvidas e cada resultado gerado é analisado e conferido em busca de erros, divergências ou detalhes importantes sobre seus requisitos.

Geralmente há a designação de um ou mais profissionais qualificados e capacitados para executar a testagem, tudo depende do porte do sistema.

Um dos principais colaboradores do desenvolvimento da Engenharia de Software, Edsger Dijkstra (1972), declarou:

"O teste só consegue mostrar a presença de erros, não a sua ausência."

Teste de Software

Quem testa um software, busca 2 coisas: Validar e Verificar.

Apesar de serem frequentemente confundidas, validar e verificar não são a mesma coisa. Barry Boehm, um pioneiro da engenharia de software nos ajuda a entender a diferença:

VALIDAR

- Estamos construindo o produto certo?



VERIFICAR

- Estamos construindo o produto corretamente?

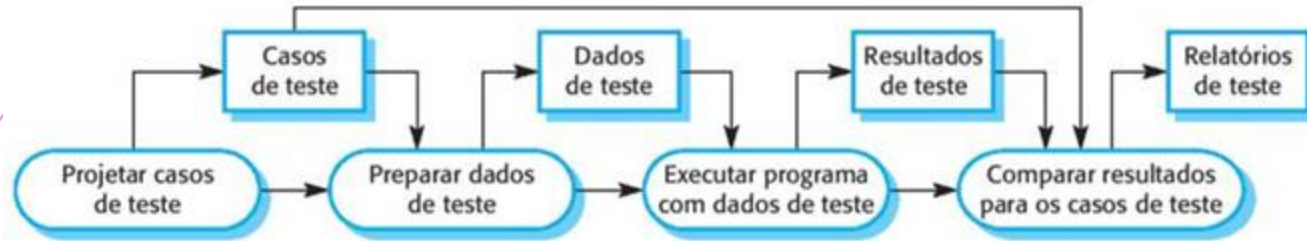
Esses 2 processos estão preocupados em conferir se o software que está sendo desenvolvido atende ao que foi planejado (especificado) e se vai entregar as funcionalidades esperadas pelos patrocinadores do projeto. Eles começam a ser “buscados” logo que o levantamento dos requisitos é concluído e disponibilizado e continuam junto com o processo de desenvolvimento.

Definindo individualmente, podemos dizer:

- ▶ A verificação é o processo de conferir se o software cumpre seus requisitos funcionais e não funcionais;
- ▶ A validação é o processo de assegurar que o software atenda às expectativas do cliente – que vai além dos requisitos funcionais e não funcionais. Lembre-se: nem sempre os requisitos refletem exatamente o que o cliente deseja e/ou precisa;

Teste de Software

Podemos definir um modelo de processo de teste de software da seguinte forma:



Geralmente, um sistema de software comercial passa por 3 estágios de teste:

- ▶ Teste de Desenvolvimento
- ▶ Teste de Lançamento (release)
- ▶ Teste de Usuário

TESTE DE DESENVOLVIMENTO

Nesta modalidade de teste, o sistema é testado durante o desenvolvimento para descobrir defeitos (*bugs*).

Geralmente, o testador é o próprio programador, ou seja, ele desenvolve e depois efetua os testes. Mas, quando necessário, o programador se junta a um testador para auxiliá-lo no processo.

Os testes de desenvolvimento podem ser divididos em 3 estágios:

- ▶ Teste de Unidade
- ▶ Teste de Componentes
- ▶ Teste de Sistema

TESTE DE DESENVOLVIMENTO

Teste de Unidade

Testa as unidades do sistema ou as classes individuais. O objetivo aqui é testar a funcionalidade dos objetos e seus métodos.



Teste de Componente

Testa as interfaces dos componentes que promovem acesso às suas funções. Várias unidades são integradas, criando componentes.



Teste de Sistema

Testa o sistema como um todo, ou seja, as interações entre os componentes.

TESTE DE DESENVOLVIMENTO

Ainda dentro da categoria de testes de desenvolvimento há uma outra variação chamada "**Desenvolvimento dirigido por testes**" (*TDD – Test Driven Development*). Nesta abordagem, desenvolvimento (programação) e testes são intercalados, ou seja, o desenvolvimento é incremental e cada bloco desenvolvido é testado e o próximo incremento do sistema só é iniciado quando o bloco anterior é aprovado.

Esse modelo de desenvolvimento faz parte das metodologias ágeis (XP) e atualmente é bastante utilizado pelo mercado.

TESTE DE LANÇAMENTO (RELEASE)

Nesta modalidade de teste, há uma equipe de testes separada que testa uma versão completa do sistema antes de ela ser entregue aos usuários finais.

O objetivo principal deste teste é verificar se o sistema cumpre os requisitos dos stakeholders.

Aqui os testes são feitos em um processo "caixa-preta" no qual o sistema é testado a partir dos documentos de especificação.

Também conhecido como Teste Funcional, pois a preocupação é com as funcionalidades, e não com a implementação.

TESTE DE LANÇAMENTO (RELEASE)

Como ramificação do Teste de Lançamento, podemos citar:

Teste baseado em requisitos

- O objetivo é demonstrar que o sistema implementou seus requisitos corretamente.

Teste de cenário

- São criados cenários de uso típicos, com o objetivo de empregá-los para desenvolver casos de teste para o sistema.

Teste de desempenho

- São projetados para garantir que o sistema possa processar sua carga pretendida.

TESTE DE USUÁRIO

Nesta modalidade de teste, os futuros usuários do sistema efetuam uma bateria de testes no seu ambiente de trabalho. O Teste de Aceitação, que é um teste de Usuário, é utilizado para formalizar a aceitação do sistema pelo cliente.

O teste de usuário é fundamental, mesmo que outros testes já tenham sido realizados, pois as influências do ambiente de trabalho do usuário final pode gerar resultados com efeito importante na confiança, na avaliação de desempenho e na usabilidade que serão atribuídas ao sistema.

Podemos dividir esse teste em três tipos diferentes: Teste Alfa, Teste Beta e Teste de Aceitação.

TESTE DE USUÁRIO

Resumidamente:

- ▶ Teste Alfa → seleciona-se um grupo de usuários para trabalhar em colaboração com os desenvolvedores e efetuar lançamentos iniciais no sistema.
- ▶ Teste Beta → seleciona-se um grupo maior de usuários e estes experimentam e informam problemas para os desenvolvedores do sistema.
- ▶ Teste de Aceitação → os usuários finais testam o sistema por completo para definir se aceitam ou não o resultado final.



8

Manutenção de Software

Definição, tipos e utilização

Manutenção de Software

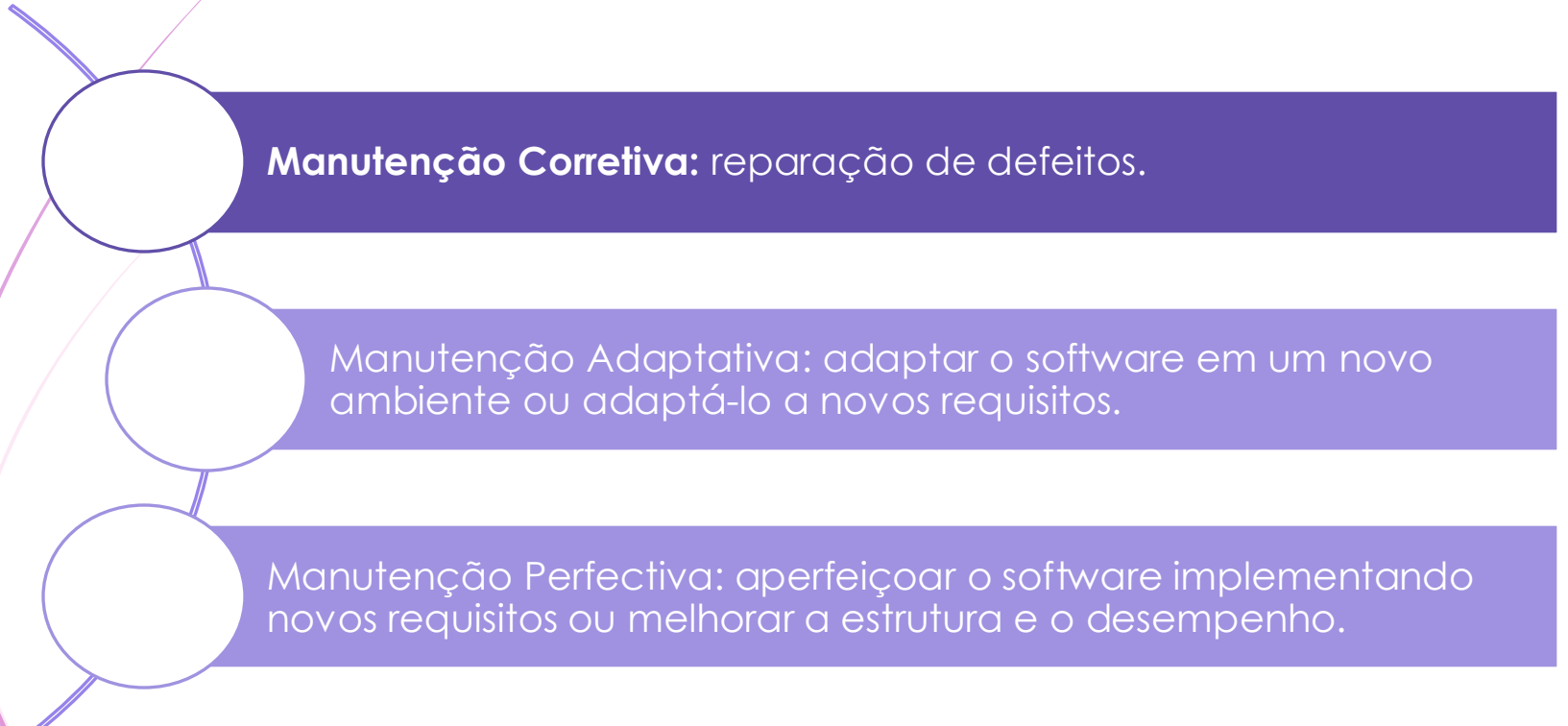
A Manutenção de Software é a etapa que se inicia após a entrega do sistema para os usuários finais. Com a utilização cotidiana do produto que foi desenvolvido vão se observando necessidades de mudança, complementações e até mesmo correções de regras de negócio ou de projeto.

Sua aplicação é mais comum em softwares personalizados e podem ser divididas em 3 tipos de manutenção:

- ▶ Reparo de defeitos para corrigir bugs e vulnerabilidades;
- ▶ Adaptação ao ambiente para adaptar o software a novas plataformas e ambientes;
- ▶ Acréscimo de funcionalidade para adicionar novas características e apoiar novos requisitos.

Manutenção de Software

Esses 3 grupos de manutenção de software, podem receber nomenclaturas específicas:



Manutenção Corretiva: reparação de defeitos.

Manutenção Adaptativa: adaptar o software em um novo ambiente ou adaptá-lo a novos requisitos.

Manutenção Perfectiva: aperfeiçoar o software implementando novos requisitos ou melhorar a estrutura e o desempenho.

Manutenção de Software

Em 2010, Davidsen e Krostie fizeram uma pesquisa sobre os custos aproximados dos serviços de manutenção. Embora não tenhamos uma pesquisa mais recente, é possível verificar que, em grande parte, a distribuição dos custos ainda é correta.

A pesquisa indica que cerca de 19% dos custos é empenhando em adaptação de ambiente, 24% com reparo de defeitos e 58% voltados para implementações ou alterações de funcionalidades.

A experiência mostra que é mais custoso implementar recursos no software durante a fase de manutenção. Sendo assim, quanto mais funcionalidades e recursos puderem ser levantados e especificados durante o desenvolvimento inicial, menos custoso será.

Reengenharia de Software

A reengenharia de software se faz necessária quando um determinado sistema, que precisa de manutenção, é muito complexo de se entender e de alterar. De modo geral, isso ocorre mais com sistemas legados mais antigos.

Podemos falar desse processo também em sistemas que já foram implementados e melhorados até os limites que sua estrutura permitem, então, para crescer mais, seria necessário recomençar.

Quando a reengenharia entra em questão podemos envolver nova documentação, refatoração de arquitetura, tradução para outra linguagem de programação e outras.

Há duas grandes vantagens que podemos citar: menor custo e menor risco.

Reengenharia de Software

Podemos elencar as seguintes atividades no processo de reengenharia de software:

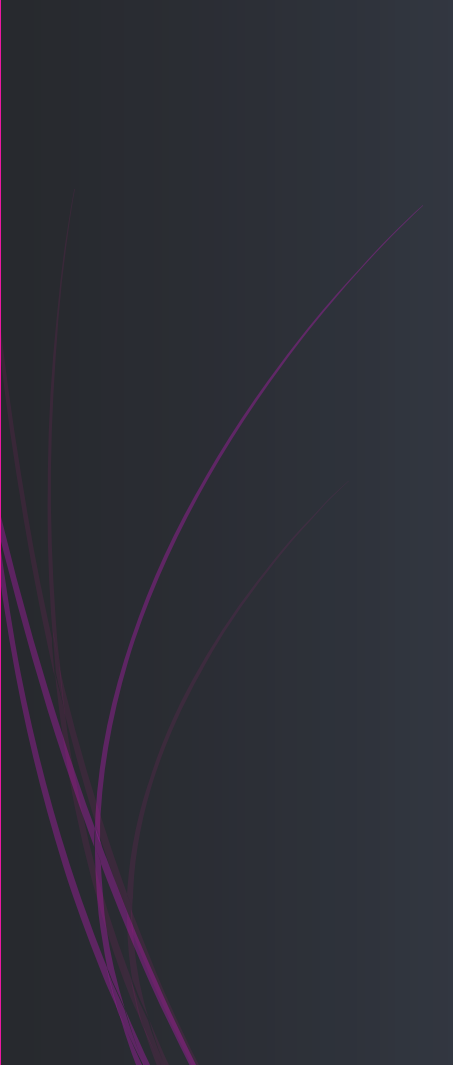
- ▶ *Tradução do código fonte*: usando um software específico, é possível traduzir uma linguagem de programação antiga, para uma versão mais moderna.
- ▶ *Engenharia reversa*: ocorre uma análise do programa e a extração de informações. Esse processo geralmente é automatizado.
- ▶ *Melhoria de estrutura do programa*: a estrutura de controle do programa é analisada para facilitar a leitura e compreensão. Há necessidade de interação
- ▶ *Modularização do programa*: com o objetivo de remover alguma redundância que for necessária. Esse processo precisa ser manual.

Reengenharia de Software

- *Reengenharia de dados*: os dados processados são modificados para refletir as alterações do programa. Também podem ocorrer limpeza de dados corrigindo erros e removendo duplicidades. Esse é um processo caro e demorado.

Processo de reengenharia:





Obrigada!

Bibliografia utilizada:

Engenharia de Software | Ian Sommerville – 10a Edição 2018